

**UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA**

TEC508 – Computação de Alto Desempenho

Docente: Angelo Amâncio Duarte

**PARALELISMO COM MEMÓRIA  
COMPARTILHADA**

*Geração do Conjunto de Mandelbrot*

Guilherme Fernandes Sardinha e Robson Carvalho de Souza

Feira de Santana – BA

2026

# Sumário

|                                                   |          |
|---------------------------------------------------|----------|
| <b>Sumário</b>                                    | <b>1</b> |
| 1 Introdução                                      | 4        |
| 2 Fundamentação Teórica                           | 5        |
| 2.1 Lei de Amdahl                                 | 5        |
| 2.2 GIL do CPython                                | 5        |
| 2.3 Metodologia PCAM                              | 5        |
| 3 Metodologia                                     | 6        |
| 3.1 Descrição das Máquinas Utilizadas             | 6        |
| 3.2 Algoritmo e Volume de Dados                   | 7        |
| 3.3 Aplicação do PCAM ao Conjunto de Mandelbrot   | 7        |
| 3.4 Otimização do Código Python Serial            | 8        |
| 3.5 Implementações Paralelas                      | 9        |
| 3.5.1 C com OpenMP                                | 9        |
| 3.5.2 Python com Multithreading                   | 9        |
| 3.5.3 Python com Multiprocessing                  | 10       |
| 4 Resultados e Análise                            | 11       |
| 4.1 time – Métrica de Tempo e Recursos do Sistema | 11       |
| 4.1.1 Máquina A                                   | 11       |
| 4.1.2 Máquina B                                   | 11       |
| 4.2 gprof – Identificação de Hotspot              | 11       |
| 4.2.1 Máquina A                                   | 11       |
| 4.2.2 Máquina B                                   | 12       |
| 4.3 perf: Métricas de Hardware                    | 12       |
| 4.3.1 Máquina A                                   | 12       |
| 4.3.2 Máquina B                                   | 14       |
| 4.4 Valgrind: Callgrind e Cachegrind              | 14       |
| 4.4.1 Máquina A                                   | 14       |
| 4.4.2 Máquina B                                   | 15       |
| 4.5 strace: Chamadas de Sistema                   | 15       |
| 4.5.1 Máquina A                                   | 15       |
| 4.5.2 Máquina B                                   | 16       |
| 4.6 Profiling da Versão Python Serial             | 16       |
| 4.6.1 Máquina A                                   | 16       |
| 4.6.1.1 cProfile                                  | 16       |
| 4.6.1.2 perf stat : Comparação com C Serial       | 17       |
| 4.6.1.3 strace : Comparação com C Serial          | 17       |

|          |                                                                |    |
|----------|----------------------------------------------------------------|----|
| 4.6.2    | Máquina B . . . . .                                            | 18 |
| 4.6.2.1  | cProfile . . . . .                                             | 18 |
| 4.6.2.2  | perf stat: Comparação com C Serial . . . . .                   | 18 |
| 4.6.2.3  | strace: Comparação com C Serial . . . . .                      | 19 |
| 4.6.3    | Máquina B . . . . .                                            | 19 |
| 4.7      | Análise da Versão C com OpenMP . . . . .                       | 19 |
| 4.7.1    | Máquina A . . . . .                                            | 19 |
| 4.7.1.1  | perf stat : Comparação com C Serial . . . . .                  | 19 |
| 4.7.1.2  | Escalabilidade: Speedup e Eficiência . . . . .                 | 20 |
| 4.7.2    | Máquina B . . . . .                                            | 22 |
| 4.7.2.1  | perf stat: Comparação com C Serial . . . . .                   | 22 |
| 4.7.2.2  | Escalabilidade: Speedup e Eficiência . . . . .                 | 23 |
| 4.8      | Análise da Versão Python com Multithreading . . . . .          | 24 |
| 4.8.1    | Máquina A . . . . .                                            | 24 |
| 4.8.1.1  | cProfile e perf . . . . .                                      | 24 |
| 4.8.1.2  | Impacto do GIL: Speedup e Eficiência . . . . .                 | 24 |
| 4.8.2    | Máquina B . . . . .                                            | 25 |
| 4.8.2.1  | cProfile . . . . .                                             | 25 |
| 4.8.2.2  | perf stat . . . . .                                            | 26 |
| 4.8.3    | Máquina B . . . . .                                            | 27 |
| 4.9      | Análise da Versão Python com Multiprocessing . . . . .         | 27 |
| 4.9.1    | Máquina A . . . . .                                            | 28 |
| 4.9.1.1  | cProfile e perf . . . . .                                      | 28 |
| 4.9.1.2  | Escalabilidade: Speedup e Eficiência . . . . .                 | 28 |
| 4.9.2    | Máquina B . . . . .                                            | 29 |
| 4.9.2.1  | cProfile e Perf . . . . .                                      | 29 |
| 4.9.2.2  | Escalabilidade: Speedup e Eficiência . . . . .                 | 30 |
| 4.9.3    | Escalabilidade: Speedup e Eficiência . . . . .                 | 30 |
| 4.10     | Overhead de Processo e Memória . . . . .                       | 31 |
| 4.10.1   | Máquina A . . . . .                                            | 31 |
| 4.10.2   | Máquina B . . . . .                                            | 32 |
| 4.11     | Tabela Comparativa dos Três Modelos de Paralelização . . . . . | 32 |
| 4.11.1   | Máquina A . . . . .                                            | 32 |
| 4.11.2   | Máquina B . . . . .                                            | 34 |
| 4.12     | Comparação C Serial versus Python Serial . . . . .             | 35 |
| 4.12.1   | Máquina A . . . . .                                            | 35 |
| 4.12.1.1 | Tempo de Execução . . . . .                                    | 35 |
| 4.12.1.2 | Métricas de Hardware . . . . .                                 | 35 |
| 4.12.1.3 | Eficiência de Paralelização . . . . .                          | 36 |

|                              |                                            |           |
|------------------------------|--------------------------------------------|-----------|
| 4.12.2                       | Máquina B . . . . .                        | 36        |
| 4.12.3                       | Tempo de Execução . . . . .                | 36        |
| 4.12.4                       | Métricas de Hardware (perf stat) . . . . . | 37        |
| 4.12.5                       | Eficiência de Paralelização . . . . .      | 37        |
| 5                            | Conclusão . . . . .                        | 38        |
| <b>Referências . . . . .</b> |                                            | <b>39</b> |

# 1 Introdução

O conjunto de Mandelbrot é um dos fractais mais conhecidos da matemática, definido no plano complexo pelo conjunto de pontos  $c$  para os quais a sequência  $z_{n+1} = z_n^2 + c$ , com  $z_0 = 0$ , não diverge. O algoritmo de geração consiste em testar, para cada ponto de uma região do plano complexo, se a sequência diverge após um número máximo de iterações, determinando assim se o ponto pertence ou não ao conjunto. Sua natureza computacional permite facilmente o paralelismo: cada ponto pode ser avaliado de forma independente, tornando-o um candidato natural para exploração de paralelismo.

No contexto de Computação de Alto Desempenho (HPC), a escolha da linguagem de programação e do modelo de paralelização tem impacto direto no desempenho obtido. Linguagens compiladas como *C* permitem ao compilador gerar código de máquina otimizado, enquanto linguagens interpretadas como *Python* introduzem camadas de abstração que impactam negativamente o tempo de execução, especialmente em cargas de trabalho intensivas em *CPU*. Compreender quantitativamente essa diferença, e como cada modelo de paralelização se comporta em cada contexto, é essencial para decisões de projeto em sistemas de alto desempenho.

## Objetivo Principal

Este trabalho visa realizar a implementação, profiling e comparação entre códigos compilados e interpretados em suas versões seriais e paralelas, utilizando um ambiente de memória compartilhada e explorando os núcleos dos processadores das máquinas locais dos integrantes da dupla.

## Objetivos Específicos

- Implementar o algoritmo de geração do conjunto de Mandelbrot nas versões *C* Serial, *C* com OpenMP, *Python* Serial, *Python* com Multithreading e *Python* com Multiprocessing;
- Aplicar a metodologia PCAM como guia de pensamento paralelo para estruturar as implementações paralelas;
- Realizar profiling de cada versão, identificando hotspots e métricas de hardware;
- Analisar a escalabilidade de cada modelo paralelo com 1, 2, 4 e 8 threads ou processos, calculando speedup e eficiência e comparando com o limite teórico da Lei de Amdahl;
- Investigar o impacto do GIL do CPython sobre a eficiência do multithreading em código CPU-bound, confrontando com o paralelismo real obtido via multiprocessing;

- Comparar quantitativamente o desempenho entre código compilado (*C*) e interpretado (*Python*), avaliando as implicações para HPC;
- Comparar os resultados obtidos nas máquinas dos dois integrantes da dupla, discutindo a influência da arquitetura de hardware sobre o desempenho e a escalabilidade.

## 2 Fundamentação Teórica

### 2.1 Lei de Amdahl

A Lei de Amdahl estabelece o speedup máximo teórico obtido ao paralelizar um algoritmo, sendo esse limite determinado pela fração serial do código. Formalmente, dado que  $p$  representa a fração paralelizável do algoritmo e  $N$  o número de processadores, o speedup máximo  $S(N)$  é definido por:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}} \quad (1)$$

Quando  $N \rightarrow \infty$ , o speedup converge para  $\frac{1}{1-p}$ , evidenciando que mesmo com recursos ilimitados, a porção serial impõe um teto absoluto ao ganho de desempenho. No contexto deste trabalho, os resultados de speedup medidos para cada modelo de paralelização são confrontados com esse limite teórico para avaliar a eficiência real das implementações.

### 2.2 GIL do CPython

O *Global Interpreter Lock* (GIL) é um *mutex* do CPython que impede que múltiplas *threads* executem bytecode simultaneamente (Python Wiki, 2024), motivado pelo fato de que o gerenciamento de memória por contagem de referências do CPython não é *thread-safe*. Isso torna o *multithreading* ineficaz para código CPU-bound, pois as *threads* disputam o lock em vez de executar em paralelo. A alternativa é o **multiprocessing**, onde cada processo possui sua própria instância do interpretador e GIL independente, permitindo paralelismo real.

### 2.3 Metodologia PCAM

O PCAM (FOSTER, 1995) é uma metodologia de projeto de algoritmos paralelos composta por quatro etapas: **Particionamento**, no qual o problema é dividido em tarefas menores para expor o máximo de paralelismo; **Comunicação**, que analisa o fluxo de dados e a coordenação entre as tarefas; **Aglomerção**, onde tarefas são agrupadas em unidades maiores para reduzir overhead de comunicação; e **Mapeamento**, que define em quais processadores cada tarefa será executada, visando minimizar ociosidade. Neste

trabalho, o PCAM guiou o projeto das três implementações paralelas, conforme detalhado na Seção 3.3.

### 3 Metodologia

Este capítulo descreve as decisões de implementação e a abordagem adotada para estruturar as versões serial e paralela do algoritmo.

Os experimentos foram executados individualmente nas máquinas dos dois integrantes da dupla, sob sistema operacional Linux, permitindo comparações tanto entre modelos de paralelização quanto entre arquiteturas de hardware distintas.

#### 3.1 Descrição das Máquinas Utilizadas

Os experimentos foram executados em dois ambientes computacionais para avaliar o impacto de diferentes topologias de hardware na escalabilidade do algoritmo. A Tabela 1 sintetiza as configurações de cada nó experimental.

Tabela 1 – Especificação das plataformas de hardware e software utilizadas.

| Componente                   | Máquina A (LOQ)          | Máquina B (Debian)       |
|------------------------------|--------------------------|--------------------------|
| Processador (CPU)            | Intel Core i5-12450HX    | Intel Core i7-11800H     |
| Arquitetura                  | Híbrida (Alder Lake)     | Monolítica (Tiger Lake)  |
| Núcleos Físicos / Lógicos    | 8 (4P + 4E) / 12         | 8 / 16                   |
| Frequência Base / Boost      | 2.40 GHz / 4.40 GHz      | 2.30 GHz / 4.60 GHz      |
| Cache L1 (Instrução / Dados) | 384 KiB / 320 KiB        | 256 KiB / 384 KiB        |
| Cache L2 / L3                | 7 MiB / 12 MiB           | 10 MiB / 24 MiB          |
| Memória RAM                  | 16 GB DDR5               | 16 GB DDR4               |
| Sistema Operacional          | Ubuntu (Kernel 7.0.0-31) | Debian (Kernel 6.1.0-52) |
| Compilador C                 | GCC 15.2.0               | GCC 12.2.0               |
| Flags de Compilação          | -O2 -fopenmp             | -O2 -fopenmp             |
| Interpretador Python         | CPython 3.14.4           | CPython 3.11.2           |

Fonte: Elaboração própria (2026).

A Máquina A utiliza a arquitetura híbrida Intel Alder Lake (INTEL CORPORATION, 2021), que combina 4 núcleos de desempenho (*P-cores*, com *Hyper-Threading*) e 4 núcleos de eficiência (*E-cores*), totalizando 12 núcleos lógicos. Essa assimetria afeta o balanceamento de carga e a divisão de trabalho entre as threads nas versões com OpenMP e Multiprocessing.

A Máquina B adota uma topologia homogênea baseada na arquitetura Intel Tiger Lake (Intel Corporation, 2021), composta por 8 núcleos físicos simétricos. Diferentemente da

Máquina A, a ausência de núcleos de eficiência garante uniformidade no balanceamento de carga entre as threads nas versões com OpenMP e Multiprocessing.

Para padronizar os ensaios e mitigar variações de frequência induzidas por *thermal throttling* ou gerenciamento dinâmico de energia, ambas as máquinas operaram conectadas à tomada sob o perfil de desempenho máximo do sistema operacional.

### 3.2 Algoritmo e Volume de Dados

A carga de trabalho foi fixada em uma grade de  $15\,360 \times 10\,240$  pontos, totalizando 157 286 400 coordenadas computadas de maneira independente. A Tabela 2 resume as constantes utilizadas em todos os ensaios.

Tabela 2 – Parâmetros de execução do algoritmo.

| Parâmetro                                  | Valor                                |
|--------------------------------------------|--------------------------------------|
| Dimensões da grade (WIDTH $\times$ HEIGHT) | $15\,360 \times 10\,240$ pixels      |
| Total de pontos                            | 157 286 400                          |
| Plano complexo ( $\Re \times \Im$ )        | $[-2, 0; 1, 0] \times [-1, 0; 1, 0]$ |
| Limite de iterações (MAX_ITER)             | 250                                  |
| Raio de escape ( $ z $ )                   | 2,0                                  |

Fonte: Os autores (2026).

A coleta experimental foi dividida em dois cenários:

- **Ensaio de profiling e métricas de hardware:** Os testes de instrumentação e contadores de desempenho (*perf*, *gprof*, *cProfile*, *Valgrind* e *strace*) foram executados de forma serial nas versões seriais e sob alocação fixa de 4 *threads* ou processos nas implementações paralelas.
- **Ensaio de escalabilidade:** Nas versões paralelas (OpenMP, Multithreading e Multiprocessing), a quantidade de unidades de execução variou entre 1, 2, 4 e 8 para o cálculo do *speedup* e eficiência, mantendo-se constante o volume de dados de entrada.

### 3.3 Aplicação do PCAM ao Conjunto de Mandelbrot

O projeto de paralelização adotou a metodologia PCAM abordada na seção 2.3. O método decompõe o problema a partir de uma granularidade fina e reestrutura a carga de processamento para balancear o trabalho entre os núcleos e reduzir sobrecargas de sincronização. No cálculo do fractal de Mandelbrot, as quatro etapas foram delineadas da seguinte forma:

- **Particionamento (*Partitioning*):** Realizou-se a decomposição de domínio em granularidade fina, na qual cada tarefa consiste no cálculo de escape de um único pixel  $(x, y)$ . Como a convergência da função  $z_{k+1} = z_k^2 + c$  depende apenas da coordenada processada, as tarefas são completamente desacopladas e possuem independência de dados.
- **Comunicação (*Communication*):** Durante a execução dos cálculos, a comunicação horizontal entre tarefas é nula, visto que um pixel não necessita de dados de pixels adjacentes. Utiliza-se comunicação puramente vertical no padrão mestre-trabalhador (*Master/Worker*), no qual a thread ou processo principal despacha o trabalho e consolida os resultados parciais preenchidos pelas unidades executoras.
- **Aglomeração (*Agglomeration*):** Escalonar e gerenciar o despacho de milhões de microtarefas individuais geraria uma sobrecarga de sincronização superior ao tempo de processamento útil. Para mitigar esse *overhead* e aproveitar a localidade espacial na hierarquia de memória cache, as tarefas elementares foram aglomeradas geometricamente em blocos contíguos de linhas da imagem.
- **Mapeamento (*Mapping*):** A carga computacional da imagem é heterogênea: linhas com grande concentração de pontos externos escapam com poucas iterações, enquanto regiões do interior do conjunto demandam máximo de iterações. O uso de escalonamento puramente estático deixaria núcleos ociosos prematuramente; dessa forma, adotou-se o mapeamento dinâmico para garantir o equilíbrio contínuo de trabalho entre as unidades de processamento.

### 3.4 Otimização do Código Python Serial

A primeira versão do código Python serial foi escrita espelhando diretamente a implementação em C, resultando em um tempo de execução superior a uma hora para a carga de trabalho definida. Após análise, o código foi otimizado com foco em três pontos: substituição da classe `Complex` por operações nativas com números de ponto flutuante, eliminando overhead de chamadas de método; aplicação de memoization para os tons dos pixels, evitando recálculos redundantes; e remoção de chamadas de função desnecessárias no laço principal. Com essas mudanças, o tempo de execução caiu para aproximadamente 11 minutos, uma redução de cerca de 5x, mantendo a restrição de não utilizar bibliotecas com implementação em C, como NumPy ou math.

## 3.5 Implementações Paralelas

### 3.5.1 C com OpenMP

A paralelização em C foi realizada com OpenMP, uma API para programação em memória compartilhada. Com o planejamento PCAM já definido (Seção 3.3), a implementação se resumiu à adição da diretiva `pragma` no laço externo de despacho de pixels:

```
1 #pragma omp parallel for schedule(dynamic, 4)
2 for (int py = 0; py < height; py++) { ... }
```

Listing 1 – Paralelização do loop de trabalho.

O escalonamento `dynamic` com chunk de 4 linhas foi escolhido para balancear a carga entre os cores, dado que linhas distintas do conjunto de Mandelbrot possuem custos computacionais variáveis. Para evitar condições de corrida nas escritas, substituiu-se as chamadas de `fputs` por um buffer alocado em RAM, com escrita do arquivo realizada uma única vez ao final da execução:

```
1 unsigned char *image = (unsigned char *) malloc(width * height * 3 *
    sizeof(unsigned char));
2
3 // Dentro do loop se avalia o pixel:
4 int idx = (py * width + px) * 3;
5 image[idx + 0] = r;
6 image[idx + 1] = g;
7 image[idx + 2] = b;
```

Listing 2 – Otimização de escrita da imagem.

### 3.5.2 Python com Multithreading

A versão concorrente emprega o módulo nativo `threading` acoplado a uma fila sincronizada (`Queue`) para distribuição dinâmica de trabalho sob a arquitetura *Master/Worker*. As threads operam no mesmo espaço de endereçamento do processo principal e escrevem diretamente em um vetor `bytearray` pré-alocado. Como as regiões de escrita em memória são mutuamente disjuntas, dispensou-se o emprego de travas por pixel. Contudo, em decorrência da natureza estritamente *CPU-bound* do algoritmo, o ganho de escalabilidade multinúcleo fica condicionado à disputa contínua pelo *Global Interpreter Lock* (GIL) no interpretador CPython.

```
1 queue = Queue()
2 for py in range(height):
3     queue.put(py)
4 for _ in range(num_threads):
5     queue.put(None)
6
```

```

7 threads = [
8     threading.Thread(target=worker_mandelbrot_line,
9                       args=(image_buffer, queue, width, height, min_x,
10                            max_x, min_y, max_y, max_iter, color_table))
11     for _ in range(num_threads)
12 ]
13 for t in threads: t.start()
14 for t in threads: t.join()

```

Listing 3 – Despacho de threads com fila sincronizada.

### 3.5.3 Python com Multiprocessing

Para contornar a serialização imposta pelo GIL e viabilizar execução paralela real, utilizou-se o módulo `multiprocessing`, orquestrado por classes `Process`, filas `Queue` e blocos de `SharedMemory`.

A opção por processos explícitos associados à memória compartilhada em detrimento de executores abstratos como o `ProcessPoolExecutor` teve como propósito mitigar a sobrecarga de comunicação interprocessos. Em estruturas convencionais de *pool*, transferir a matriz final de  $15\,360 \times 10\,240 \times 3$  bytes demandaria serializações contínuas via módulo `pickle` através de canais *pipe* do sistema operacional. A alocação de um segmento compartilhado *POSIX* permite que os processos filhos registrem os valores de cor diretamente no buffer por meio de ponteiros nativos, extinguindo cópias redundantes em memória. Definiu-se ainda a estratégia de instanciação via `fork` para minimizar a latência de inicialização dos processos no Linux.

```

1 total_bytes = width * height * 3
2 shm = SharedMemory(create=True, size=total_bytes)
3
4 queue = Queue()
5 for py in range(height):
6     queue.put(py)
7 for _ in range(num_processos):
8     queue.put(None)
9
10 processos = [
11     Process(target=worker_mandelbrot_line,
12            args=(shm.name, queue, width, height, min_x, max_x, min_y,
13                 max_y, max_iter, color_table))
14     for _ in range(num_processos)
15 ]
16 for p in processos: p.start()
17 for p in processos: p.join()

```

Listing 4 – Alocação de memória compartilhada e instanciação de processos.

## 4 Resultados e Análise

Esta seção apresenta a avaliação experimental das cinco versões desenvolvidas para o cálculo do Conjunto de Mandelbrot. As análises estruturam-se na extração de métricas de tempo e uso de recursos via `/usr/bin/time -v,profiling` com contadores de hardware (`perf`), mapeamento de gargalos com ferramentas de instrumentação (`gprof`, `Valgrind`, `strace` e `cProfile`) e estudo de escalabilidade e eficiência dos modelos concorrentes.

### 4.1 time – Métrica de Tempo e Recursos do Sistema

#### 4.1.1 Máquina A

O comportamento temporal e a alocação de recursos das cinco versões foram monitorados via GNU `/usr/bin/time -v` sob a resolução de  $15360 \times 10240$  pixels e 250 iterações máximas. Para os modelos paralelos, foi analisado a versão com 4 *workers*, confrontados diretamente com as respectivas linhas de base sequenciais.

Tabela 3 – Tabela Comparativa — `/usr/bin/time -v` (Máquina B, 4 workers nas versões paralelas)

| Versão            | Wall-clock | User time | Sys time | % CPU | RSS máx (KB) | Page faults (maj / min) | Ctx switches (vol / invol) | Perfil    |
|-------------------|------------|-----------|----------|-------|--------------|-------------------------|----------------------------|-----------|
| C Serial          | 25,75 s    | 25,52 s   | 0,21 s   | 99%   | 2.544        | 0 / 98                  | 6 / 882                    | CPU-bound |
| C OpenMP (4T)     | 6,57 s     | 25,37 s   | 0,32 s   | 391%  | 463.004      | 0 / 115.321             | 10 / 347                   | CPU-bound |
| Python Serial     | 8:28 min   | 508,40 s  | 0,13 s   | 99%   | 10.724       | 1 / 1.068               | 27 / 13.123                | CPU-bound |
| Py Threading (4T) | 8:39 min   | 520,66 s  | 3,58 s   | 100%  | 472.644      | 0 / 116.528             | 551.321 / 9.829            | CPU-bound |
| Py Multiproc (4P) | 2:31 min   | 604,81 s  | 0,62 s   | 399%  | 942.640      | 0 / 253.892             | 452 / 17.030               | CPU-bound |

#### 4.1.2 Máquina B

Tabela 4 – Tabela Comparativa — `/usr/bin/time -v` (Máquina B, 4 workers nas versões paralelas)

| Versão            | Wall-clock | User time | Sys time | % CPU | RSS máx (KB) | Page faults (maj / min) | Ctx switches (vol / invol) | Perfil    |
|-------------------|------------|-----------|----------|-------|--------------|-------------------------|----------------------------|-----------|
| C Serial          | 31,69 s    | 31,51 s   | 0,17 s   | 99%   | 2.148        | 0 / 96                  | 3 / 206                    | CPU-bound |
| C OpenMP (4T)     | 8,08 s     | 31,68 s   | 0,19 s   | 394%  | 462.728      | 1 / 886                 | 5 / 98                     | CPU-bound |
| Python Serial     | 11:28 min  | 687,83 s  | 0,18 s   | 99%   | 10.212       | 68 / 1.102              | 102 / 5.025                | CPU-bound |
| Py Threading (4T) | 11:12 min  | 675,98 s  | 5,45 s   | 101%  | 474.296      | 0 / 2.037               | 741.045 / 1.082            | CPU-bound |
| Py Multiproc (4P) | 3:28 min   | 830,74 s  | 0,50 s   | 399%  | 939.184      | 0 / 149.740             | 10.534 / 3.363             | CPU-bound |

### 4.2 gprof – Identificação de Hotspot

#### 4.2.1 Máquina A

Ao executar a versão C serial, o *profiling* via `gprof` identificou a função `check_mandelbrot` como o gargalo computacional (*hotspot*).

Tabela 5 – Flat Profile obtido via `gprof` para a versão C Serial na Máquina A.

| Função                        | % Tempo | Cumulativo (s) | Self (s) | Chamadas    | ns/call |
|-------------------------------|---------|----------------|----------|-------------|---------|
| <code>check_mandelbrot</code> | 98,26   | 23,66          | 23,66    | 157.286.400 | 150,43  |
| <code>main</code>             | 1,04    | 23,91          | 0,25     | –           | –       |
| <code>_init</code>            | 0,71    | 24,08          | 0,17     | –           | –       |

Fonte: Elaboração própria (2026).

Os dados revelam que 98,26% do tempo total (23,66 s) é consumido por esta rotina, invocada 157.286.400 vezes uma por pixel. O *Call Graph* confirma a transferência quase total (99,3%) do tempo da função `main` para a sub-rotina. Essa alta concentração de processamento em um único laço fechado, associada ao baixo tempo por chamada (150,43 ns), evidencia uma granularidade excelente e um cenário ideal para futura paralelização de dados.

#### 4.2.2 Máquina B

O profiling via `gprof` identificou a função `check_mandelbrot` como hotspot, concentrando 98,41% do tempo total de execução (29,80 s de 30,32 s), invocada 157.286.400 vezes — exatamente um por pixel da imagem gerada.

Tabela 6 – Flat Profile obtido via `gprof` para a versão C Serial na Máquina B.

| Função                        | % Tempo | Cumulativo (s) | Self (s) | Chamadas    | ns/call |
|-------------------------------|---------|----------------|----------|-------------|---------|
| <code>check_mandelbrot</code> | 98,41   | 29,80          | 29,80    | 157.286.400 | 189,46  |
| <code>main</code>             | 1,26    | 30,18          | 0,38     | –           | –       |
| <code>_init</code>            | 0,40    | 30,30          | 0,12     | –           | –       |
| <code>mandelbrot</code>       | 0,07    | 30,32          | 0,02     | –           | –       |

Fonte: Elaboração própria (2026).

O Call Graph ?? confirma que 99,5% do tempo total da `main` foi transferido integralmente para `check_mandelbrot`. O tempo por chamada de 189,46 ns é ligeiramente superior ao registrado na Máquina A (150,43 ns), reflexo da diferença de geração rquitetural entre os processadores.

### 4.3 perf: Métricas de Hardware

#### 4.3.1 Máquina A

O monitoramento via `perf stat` revelou que 99,71% do tempo de processamento foi alocado nos núcleos de performance (*P-cores*) da arquitetura híbrida Alder Lake da Máquina A.

O IPC de 2,03 reflete o alto paralelismo em nível de instrução (ILP), favorecido pela baixa taxa de falhas de desvio (0,19%) e pela reduzida perda de cache L1/LLC. O baixo

Tabela 7 – Métricas globais de hardware extraídas via `perf stat` (P-cores + E-cores).

| Métrica                                             | Valor Consolidado    |
|-----------------------------------------------------|----------------------|
| Instruções                                          | 358.987.281.774      |
| Ciclos                                              | 182.313.143.618      |
| Instruções por Ciclo (IPC Médio)                    | 1,97                 |
| Desvios ( <i>Branches</i> )                         | 43.983.956.115       |
| Falhas de Desvio ( <i>Branch-misses</i> )           | 107.908.429 (0,25%)  |
| Acessos à Cache ( <i>Cache-references</i> )         | 245.566.916          |
| Falhas de Cache ( <i>Cache-misses</i> )             | 131.336.910 (53,48%) |
| Falhas de Cache L1 ( <i>L1-dcache-load-misses</i> ) | 13.179.830           |
| Falhas de LLC ( <i>LLC-load-misses</i> )            | 487.832              |
| Trocas de Contexto ( <i>Context-switches</i> )      | 921                  |
| Migrações de CPU ( <i>CPU-migrations</i> )          | 66                   |

número de trocas de contexto (921) e migrações de CPU (66) confirma que o processo executou de forma contínua e estável nos *P-cores*, caracterizando uma carga estritamente *CPU-bound*.

Tabela 8 – Perfil de execução e amostragem de ciclos extraídos via `perf report` (*P-cores*).

| Símbolo / Função                         | Módulo / Objeto | Overhead (Self) | Overhead (Children) |
|------------------------------------------|-----------------|-----------------|---------------------|
| <code>check_mandelbrot</code>            | programa        | 92,90%          | 93,17%              |
| <code>main</code>                        | programa        | 1,25%           | 6,52%               |
| <code>fputc</code>                       | libc.so.6       | 2,16%           | 2,58%               |
| <code>__sin_fma</code>                   | libm.so.6       | 1,83%           | 1,94%               |
| Chamadas de Sistema ( <i>I/O Write</i> ) | libc / Kernel   | ≈ 0,01%         | 0,95%               |

A análise de *profiling* via `perf report` evidencia que o núcleo de processamento matemático (`check_mandelbrot`) consumiu diretamente 92,90% dos ciclos de CPU da aplicação. As operações de formatação e escrita gráfica (`fputc`, `__sin_fma` e chamadas de sistema) somam menos de 6% do tempo total, confirmando que o algoritmo gasta quase a totalidade de seus ciclos no laço aritmético de convergência dos pontos.

### 4.3.2 Máquina B

Tabela 9 – Métricas de hardware extraídas via perf stat para a versão C Serial na Máquina B.

| Métrica                          | Valor              |
|----------------------------------|--------------------|
| Instruções                       | 221.151.417.934    |
| Ciclos                           | 145.518.217.338    |
| Instruções por Ciclo (IPC)       | 1,52               |
| Desvios (Branches)               | 26.198.428.394     |
| Falhas de Desvio (Branch-misses) | 50.719.292 (0,19%) |
| Referências de Cache             | 11.061.059         |
| Falhas de Cache (cache-misses)   | 8.850.753 (80,01%) |
| Falhas de Cache L1               | 16.882.811         |
| Falhas de LLC                    | 363.149            |
| Context-switches                 | 354                |
| CPU-migrations                   | 11                 |

Fonte: Elaboração própria (2026).

O IPC de 1,52 garante boa vazão no despacho de instruções, embora inferior ao registrado na Máquina A (2,03). O preditor de desvios comportou-se de forma favorável, com taxa de falha de apenas 0,19%, idêntica à Máquina A, confirmando que o padrão do algoritmo é previsível e inerente à plataforma.

A taxa de 80,01% de falhas sobre as referências de cache é irrisória: os 11 milhões de acessos representam apenas 0,004977% das instruções executadas, confirmando que o laço interno opera quase integralmente sobre registradores. Os 363.149 LLC-load-misses correspondem a apenas 0,000164% das instruções totais, consolidando o perfil estritamente CPU-bound da aplicação.

A amostragem via `perf record`, confirma os resultados do `gprof`, atribuindo 94,41% das amostras à função `check_mandelbrot`, confirmando-a como o único hotspot do algoritmo.

## 4.4 Valgrind: Callgrind e Cachegrind

Para análise com Valgrind, a entrada de dados foi reduzida utilizando resolução de 768 x 432 e 100 iterações.

### 4.4.1 Máquina A

O *profiling* dinâmico via Valgrind quantificou o comportamento do algoritmo na hierarquia de memória, sumarizado na Tabela 10.

A rotina `check_mandelbrot` concentra 78,05% das instruções (Ir), mas registrou apenas 1 falha na cache de dados L1 (D1mr) em mais de 663 mil leituras. Essa proporção indica

Tabela 10 – *profiling* de instruções e falhas de cache (Valgrind) para a versão C Serial.

| Função                   | Instruções (Ir)    | % Ir       | Leituras (Dr)     | Falhas L1 (D1mr) |
|--------------------------|--------------------|------------|-------------------|------------------|
| check_mandelbrot         | 186.966.762        | 78,05      | 663.552           | 1                |
| fputc                    | 21.896.487         | 9,14       | 6.967.053         | 0                |
| __sin_fma                | 15.240.929         | 6,36       | 4.684.411         | 61               |
| <b>Total do Programa</b> | <b>239.562.464</b> | <b>100</b> | <b>16.263.035</b> | <b>1.741</b>     |

que a carga computacional é resolvida majoritariamente nos registradores da CPU. No escopo global, o programa obteve uma taxa de acerto na L1 superior a 99,98% (1.741 falhas em 16,2 milhões de acessos).

Como o laço interno manipula apenas tipos numéricos nativos, o *footprint* de memória é mínimo e acomoda-se integralmente na L1 (32 KiB). Essa ausência de tráfego para a memória principal (RAM) comprova a alta localidade espacial e temporal do algoritmo. O cenário garante que a paralelização via OpenMP escalará de forma eficiente, processando dados isoladamente nos núcleos sem saturar o barramento de memória.

#### 4.4.2 Máquina B

Tabela 11 – Profiling de instruções e falhas de cache (Valgrind) para a versão C Serial na Máquina B.

| Função            | Instruções (Ir) | % Ir  | Leituras (Dr) | Falhas L1 (D1mr) |
|-------------------|-----------------|-------|---------------|------------------|
| check_mandelbrot  | 185.725.224     | 79,08 | 663.552       | 0                |
| fputc             | 19.906.317      | 8,48  | 6.967.053     | 0                |
| __sin_fma         | 15.042.077      | 6,40  | 4.684.411     | 60               |
| main              | 11.511.088      | 4,90  | 2.400.457     | 0                |
| Total do Programa | 234.858.739     | 100   | 16.254.389    | 1.465            |

Fonte: Elaboração própria (2026).

A rotina `check_mandelbrot` concentra 79,08% das instruções totais com zero falhas L1 em mais de 663 mil leituras, confirmando operação quase integral sobre registradores. Globalmente, o programa atingiu taxa de acerto L1 superior a 99,99% (1.465 falhas em 16,2 milhões de acessos), sem falhas de LLC nas funções principais. O laço interno acomoda-se integralmente na D1 de 48 KiB da Máquina B, com os 60 misses residuais concentrados em `__sin_fma`, função secundária de colorização de pixel.

## 4.5 strace: Chamadas de Sistema

### 4.5.1 Máquina A

O rastreamento via `strace` monitorou as requisições ao *kernel* do sistema operacional, sintetizadas na Tabela 12.

Tabela 12 – Principais chamadas de sistema (strace) da versão C Serial.

| <b>Syscall</b> | <b>% Tempo</b> | <b>Tempo (s)</b> | <b>Chamadas</b> | <b>Erros</b> |
|----------------|----------------|------------------|-----------------|--------------|
| close          | 68,21          | 0,1117           | 4               | 0            |
| openat         | 19,02          | 0,0311           | 4               | 0            |
| write          | 12,77          | 0,0209           | 115.210         | 0            |
| <b>Total</b>   | <b>100,00</b>  | <b>0,1638</b>    | <b>115.255</b>  | <b>1</b>     |

O tempo total em modo *kernel* foi irrisório (0,16 s). O volume elevado de chamadas `write` deve-se exclusivamente à gravação da imagem PPM em disco. Como não há interferência do sistema operacional durante o laço do fractal, a execução ocorre quase integralmente em espaço de usuário (*user space*), confirmando a natureza estritamente CPU-Bound do código.

#### 4.5.2 Máquina B

Tabela 13 – Principais chamadas de sistema (strace) da versão C Serial na Máquina B.

| <b>Syscall</b> | <b>% Tempo</b> | <b>Tempo (s)</b> | <b>Chamadas</b> | <b>Erros</b> |
|----------------|----------------|------------------|-----------------|--------------|
| openat         | 50,16          | 0,038314         | 4               | 0            |
| close          | 31,46          | 0,024029         | 4               | 0            |
| write          | 18,37          | 0,014028         | 115.210         | 0            |
| Total          | 100,00         | 0,076380         | 115.255         | 1            |

Fonte: Elaboração própria (2026).

O comportamento foi análogo ao da Máquina A, com tempo total em modo kernel de 0,076 s e as três syscalls predominantes sendo `openat`, `close` e `write`, conforme Tabela 13.

## 4.6 Profiling da Versão Python Serial

A versão em Python serial foi submetida a *profiling* para quantificar os custos de interpretação de *bytecode*, chamadas de sistema e uso de recursos de hardware sob a mesma carga da versão compilada.

### 4.6.1 Máquina A

#### 4.6.1.1 cProfile

O *profiling* via `cProfile` mapeou a distribuição temporal das rotinas no interpretador, resumizada na Tabela 14.

A função `check_mandelbrot` concentra 85,90% do tempo total (499,04 s de 580,97 s). O volume superior a 471 milhões de chamadas dinâmicas expõe a sobrecarga (*overhead*)

Tabela 14 – *Profiling* de execução via `cProfile` para a versão Python Serial.

| Função                          | Chamadas           | Self Time (s) | % Self        | Cumulativo (s) |
|---------------------------------|--------------------|---------------|---------------|----------------|
| <code>check_mandelbrot</code>   | 157.286.400        | 499,04        | 85,90         | 499,04         |
| <code>generate_image</code>     | 1                  | 38,20         | 6,58          | 580,97         |
| <code>mandelbrot_pixel</code>   | 157.286.400        | 30,73         | 5,29          | 529,77         |
| <code>extend (bytearray)</code> | 157.286.400        | 12,80         | 2,20          | 12,80          |
| <b>Total da Execução</b>        | <b>471.870.437</b> | <b>580,97</b> | <b>100,00</b> | <b>580,97</b>  |

de criação e destruição contínua de quadros de pilha (*stack frames*) no interpretador, custo inexistente na versão compilada.

#### 4.6.1.2 `perf stat` : Comparação com C Serial

A coleta via `perf stat` (99,97% em núcleos `cpu_core`) contrasta o uso do hardware entre ambas as linguagens, conforme a Tabela 15.

Tabela 15 – Comparação de métricas de hardware entre C Serial e Python Serial (P-cores).

| Métrica              | C Serial           | Python Serial       | Razão (Py / C) |
|----------------------|--------------------|---------------------|----------------|
| Instruções           | 224.565.189.652    | 12.010.044.786.313  | 53,48×         |
| Ciclos               | 110.761.726.131    | 2.231.523.372.434   | 20,15×         |
| IPC                  | 2,03               | 5,38                | 2,65×          |
| Branch-misses        | 50.307.704 (0,19%) | 97.076.250 (0,005%) | 1,93×          |
| L1-dcache-misses     | 13.179.830         | 172.043.178         | 13,05×         |
| LLC-load-misses      | 443.233            | 1.393.458           | 3,14×          |
| Tempo Wall-clock (s) | 24,08              | 509,62              | 21,16×         |

A versão em Python demandou 53,48 vezes mais instruções devido ao laço de avaliação de *bytecode* do CPython, tornando a execução 21,16 vezes mais lenta. O IPC *Instructions Per Cycle* de 5,38 é ilusório. Ele resulta da rápida execução de instruções repetitivas do próprio interpretador CPython ao decodificar a máquina virtual, e não de avanço aritmético útil do fractal.. Além disso, as falhas de cache L1 cresceram 13 vezes, reflexo do uso de estruturas `PyObject`, cujos metadados adicionais aumentam a ocupação da memória e fragmentam os acessos em relação aos tipos primitivos do C.

#### 4.6.1.3 `strace` : Comparação com C Serial

O perfil de chamadas de sistema (`strace`) sintetiza as diferenças operacionais de cada ambiente na Tabela 16.

O C Serial concentrou chamadas em gravações frequentes via `write`, enquanto o Python minimizou requisições de escrita agrupando dados em blocos com `bytearray`. Contudo, o interpretador gerou maior variedade de chamadas (35 contra 18), dedicando 67,02% do tempo em *kernel* a operações de inicialização, localização de módulos (`openat`) e alocação de memória (`mmap` e `brk`).

Tabela 16 – Comparação de chamadas de sistema (**strace**) entre C Serial e Python Serial.

| <b>Característica</b>       | <b>C Serial</b>               | <b>Python Serial</b>           |
|-----------------------------|-------------------------------|--------------------------------|
| Tempo Total em Kernel (s)   | 0,1638                        | 0,0285                         |
| Chamadas <b>write</b>       | 115.210                       | 5.129                          |
| Syscall Predominante        | <b>close</b> (68,21%)         | <b>openat</b> (67,02%)         |
| Total de Syscalls Distintas | 18                            | 35                             |
| Operações de Memória/SO     | 12 <b>mmap</b> , 3 <b>brk</b> | 30 <b>mmap</b> , 15 <b>brk</b> |

## 4.6.2 Máquina B

### 4.6.2.1 cProfile

Tabela 17 – Profiling de execução via cProfile para a versão Python Serial na Máquina B.

| <b>Função</b>      | <b>Chamadas</b> | <b>Self Time (s)</b> | <b>% Self</b> | <b>Cumulativo (s)</b> |
|--------------------|-----------------|----------------------|---------------|-----------------------|
| check_mandelbrot   | 157.286.400     | 1440,636             | 92,93         | 1440,636              |
| mandelbrot_pixel   | 157.286.400     | 53,518               | 3,45          | 1494,154              |
| generate_image     | 1               | 46,847               | 3,02          | 1550,214              |
| extend (bytearray) | 157.286.400     | 8,959                | 0,58          | 8,959                 |
| Total da Execução  | 471.870.437     | 1550,215             | 100,00        | 1550,215              |

Fonte: Elaboração própria (2026).

A função **check\_mandelbrot** concentra 92,93% do tempo total (1440,636 s de 1550,215 s), invocada 157.286.400 vezes — padrão estruturalmente idêntico ao da Máquina A e ao hotspot identificado pelo gprof na versão C. O volume de chamadas dinâmicas apresentou comportamento idêntico ao da Máquina A.

### 4.6.2.2 perf stat: Comparação com C Serial

Tabela 18 – Comparação de métricas de hardware entre C Serial e Python Serial na Máquina B.

| <b>Métrica</b>       | <b>C Serial</b>    | <b>Python Serial</b> | <b>Razão (Py / C)</b> |
|----------------------|--------------------|----------------------|-----------------------|
| Instruções           | 221.151.417.934    | 12.174.988.295.716   | 55,04×                |
| Ciclos               | 145.518.217.338    | 3.287.253.055.502    | 22,59×                |
| IPC                  | 1,52               | 3,70                 | 2,43×                 |
| Branch-misses        | 50.719.292 (0,19%) | 136.271.051 (0,01%)  | 2,69×                 |
| L1-dcache-misses     | 16.882.811         | 77.734.815           | 4,60×                 |
| LLC-load-misses      | 363.149            | 270.392              | 0,74×                 |
| Tempo Wall-clock (s) | 31,86              | 734,83               | 23,06×                |

Fonte: Elaboração própria (2026).

A versão Python serial demandou 55,04 vezes mais instruções que a versão C, com a execução 23,06 vezes mais lenta. O IPC superior (3,70 x 1,52) refere-se à execução de instruções de decodificação do bytecode, não de cômputo útil. A menor taxa de branch-misses em Python (0,01% contra 0,19%) decorre da previsibilidade das chamadas de abstração do interpretador. O comportamento segue o mesmo padrão da Máquina A.

#### 4.6.2.3 strace: Comparação com C Serial

#### 4.6.3 Máquina B

Tabela 19 – Comparação de chamadas de sistema (strace) entre C Serial e Python Serial na Máquina B.

| Característica                | C Serial        | Python Serial  |
|-------------------------------|-----------------|----------------|
| Tempo Total em Kernel (s)     | 0,076           | 0,031          |
| Syscall mais frequente        | write (115.210) | write (10.250) |
| 2 <sup>a</sup> mais frequente | newfstatat (5)  | read (49)      |
| 3 <sup>a</sup> mais frequente | openat (4)      | close (40)     |
| Total de Syscalls Distintas   | 18              | 32             |

Fonte: Elaboração própria (2026).

Em ambas as versões **write** domina o volume de chamadas, porém o C realiza 115.210 escritas individuais pixel a pixel contra 10.250 do Python, que agrupa os dados via **bytearray**. O Python apresenta maior variedade de syscalls (32 contra 18), reflexo do overhead de inicialização do interpretador e localização de módulos, ausente na versão compilada.

### 4.7 Análise da Versão C com OpenMP

A paralelização em memória compartilhada via diretivas OpenMP foi avaliada para quantificar o ganho de vazão computacional e a utilização dos núcleos físicos em comparação ao modelo serial.

#### 4.7.1 Máquina A

##### 4.7.1.1 perf stat : Comparação com C Serial

O monitoramento via **perf stat** com 4 *threads* (Tabela 20) avalia o comportamento da carga paralela nos núcleos de performance (**cpu\_core**), que concentraram 98,97% da atividade.

A contagem de instruções e o IPC (1,96) mantiveram-se consistentes com o perfil serial, confirmando sobrecarga de controle inerente pelas diretivas OpenMP. O salto de 13,33×

Tabela 20 – Comparação de métricas de hardware entre C Serial e C OpenMP (4 threads nos *P-cores*).

| Métrica                                        | C Serial           | C OpenMP (4T)      | Razão Relativa |
|------------------------------------------------|--------------------|--------------------|----------------|
| Instruções                                     | 224.565.189.652    | 211.512.459.872    | 0,94×          |
| Ciclos                                         | 110.761.726.131    | 107.952.000.241    | 0,97×          |
| Instruções por Ciclo (IPC)                     | 2,03               | 1,96               | 0,97×          |
| Desvios ( <i>Branches</i> )                    | 26.349.188.584     | 23.826.684.867     | 0,90×          |
| Falhas de Desvio ( <i>Branch-misses</i> )      | 50.307.704 (0,19%) | 48.812.375 (0,20%) | 0,97×          |
| Acessos à Cache ( <i>Cache-references</i> )    | 22.947.579         | 38.798.358         | 1,69×          |
| Falhas de Cache ( <i>Cache-misses</i> )        | 11.808.914         | 26.574.857         | 2,25×          |
| Falhas L1D ( <i>L1-dcache-load-misses</i> )    | 13.179.830         | 8.115.368          | 0,62×          |
| Falhas de LLC ( <i>LLC-load-misses</i> )       | 443.233            | 5.910.083          | 13,33×         |
| Trocas de Contexto ( <i>Context-switches</i> ) | 921                | 1.543              | 1,68×          |
| Migrações de CPU ( <i>CPU-migrations</i> )     | 66                 | 108                | 1,64×          |
| Tempo Wall-clock (s)                           | 27,17              | 6,87               | 0,25×          |

nas falhas de LLC deve-se à contenção de linhas de cache compartilhadas durante a escrita simultânea das *threads* na memória de saída.

O aumento em trocas de contexto (de 921 para 1.543) e migrações de CPU (de 66 para 108) decorre da sincronização das *threads* na biblioteca `libgomp` e do escalonamento do kernel entre os núcleos. Em números absolutos, esses valores continuam baixíssimos diante de mais de 107 bilhões de ciclos, comprovando que não houve preempção involuntária excessiva nem perda de afinidade.

Tabela 21 – Perfil de execução e amostragem de ciclos extraídos via `perf report` para C OpenMP (*P-cores*).

| Símbolo / Função                        | Módulo / Objeto | Self (%) | Children (%) |
|-----------------------------------------|-----------------|----------|--------------|
| <code>check_mandelbrot</code>           | programa        | 95,29%   | 95,56%       |
| <code>main._omp_fn.0</code>             | programa        | 1,17%    | 3,47%        |
| <code>__sin_fma</code>                  | libm.so.6       | 1,85%    | 1,95%        |
| <code>main</code>                       | programa        | 0,00%    | 1,09%        |
| Chamadas de Sistema ( <i>write/fs</i> ) | kernel.kallsyms | 0,00%    | 1,09%        |

O perfilamento via `perf report` corrobora que a função aritmética `check_mandelbrot` absorveu 95,29% dos ciclos em execução paralela (frente a 92,90% no serial). Isso demonstra que o código despachado por `main._omp_fn.0` concentrou o tempo quase integralmente no cálculo numérico, com o *overhead* do runtime paralelo e de I/O permanecendo em patamares marginais.

#### 4.7.1.2 Escalabilidade: Speedup e Eficiência

A avaliação de escalabilidade com 1, 2, 4 e 8 *threads*, ilustrada na Figura 1, mensura o aproveitamento real do paralelismo frente ao custo de hardware.

Em Computação de Alto Desempenho, a aceleração ( $S_p = T_1/T_p$ ) reflete apenas a redução dimensional do tempo de execução frente à linha de base serial ( $T_1$ ), ao passo

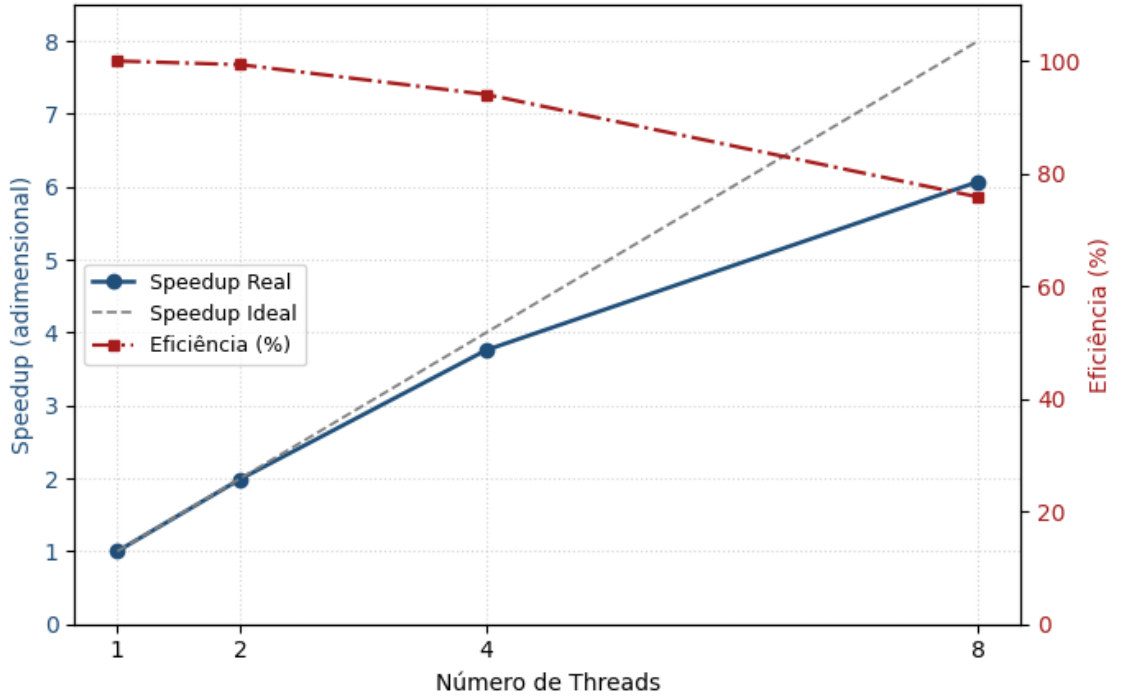


Figura 1 – Curvas de *Speedup* e Eficiência da versão C OpenMP.

que a eficiência ( $E_p = S_p/p$ ) determina a qualidade arquitetural ao quantificar a fração de trabalho útil efetivamente convertida por cada núcleo alocado ( $p$ ). A Tabela 22 sintetiza os resultados empíricos em comparação ao teto analítico da Lei de Amdahl (eq. 1), adotando a fração paralelizável de 98,26% ( $P = 0,9826$ ) mapeada no *profiling*.

Tabela 22 – Métricas de escalabilidade e comparação com a Lei de Amdahl.

| <i>Threads</i> ( $p$ ) | Tempo ( $T_p$ ) | <i>Speedup</i> Real ( $S_p$ ) | Amdahl ( $S_{Amdahl}$ ) | Eficiência ( $E_p$ ) |
|------------------------|-----------------|-------------------------------|-------------------------|----------------------|
| 1                      | 24,72 s         | 1,00×                         | 1,00×                   | 100,00%              |
| 2                      | 12,44 s         | 1,99×                         | 1,97×                   | 99,35%               |
| 4                      | 6,57 s          | 3,76×                         | 3,80×                   | 94,06%               |
| 8                      | 4,07 s          | 6,07×                         | 7,11×                   | 75,86%               |

Até 4 *threads*, o ganho aproxima-se do ideal analítico devido à independência de cálculo entre pixels e à alocação exclusiva nos núcleos de alta performance (*P-cores*). Para 8 *threads*, a eficiência sofre retração de 18,20 pontos percentuais, afastando-se da projeção teórica (6,07× contra 7,11×). Essa perda marginal decorre da assimetria microarquitetural do processador i5-12450HX da Máquina A: ao extrapolar os 4 *P-cores*, as tarefas remanescentes competem nos núcleos de eficiência (*E-cores*), os quais operam sob menor frequência e menor vazão de instruções, quebrando a premissa de simetria homogênea da Lei de Amdahl.

## 4.7.2 Máquina B

### 4.7.2.1 perf stat: Comparação com C Serial

Tabela 23 – Comparação de métricas de hardware entre C Serial e C OpenMP (4 threads) na Máquina B.

| Métrica              | C Serial           | C OpenMP (4T)       | Razão Relativa |
|----------------------|--------------------|---------------------|----------------|
| Instruções           | 221.151.417.934    | 209.898.652.822     | 0,95×          |
| Ciclos               | 145.518.217.338    | 143.023.195.915     | 0,98×          |
| IPC                  | 1,52               | 1,47                | 0,97×          |
| Referências de Cache | 11.061.059         | 35.257.617          | 3,19×          |
| Falhas de Cache      | 8.850.753 (80,01%) | 26.688.325 (75,70%) | 3,02×          |
| Branch-misses        | 50.719.292 (0,19%) | 49.886.842 (0,21%)  | 0,98×          |
| L1-dcache-misses     | 16.882.811         | 44.386.518          | 2,63×          |
| LLC-load-misses      | 363.149            | 6.106.522           | 16,81×         |
| Context-switches     | 354                | 85                  | 0,24×          |
| CPU-migrations       | 11                 | 1                   | 0,09×          |
| Tempo Wall-clock (s) | 31,86              | 8,11                | 0,25×          |

Fonte: Elaboração própria (2026).

O IPC manteve-se estável (1,47 contra 1,52 no serial). O aumento de LLC-load-misses (16,81×) decorre do acesso concorrente das 4 threads ao mesmo espaço de endereçamento da matriz de pixels. Os context-switches e CPU-migrations reduziram expressivamente (0,24× e 0,09×), evidenciando que cada thread permaneceu fixada em seu núcleo durante a execução, preservando a localidade de cache e confirmando o bom comportamento do escalonador do Linux para cargas CPU-bound.

Tabela 24 – Principais funções identificadas via perf record no C OpenMP (Máquina B).

| Self (%) | Função           | Origem            |
|----------|------------------|-------------------|
| 96,35%   | check_mandelbrot | programa          |
| 1,82%    | __sin_fma        | libm.so.6         |
| 0,96%    | main._omp_fn.0   | programa (OpenMP) |
| 0,08%    | libgomp          | libgomp.so        |

Fonte: Elaboração própria (2026).

A amostragem via `perf record` confirmou `check_mandelbrot` como hotspot com 96,35% das amostras, valor praticamente idêntico ao da versão serial. A função `main._omp_fn.0` (1,27%), injetada pelo runtime OpenMP para gerenciar a região paralela, representa o overhead das diretivas de paralelização — com impacto mínimo frente ao volume de dados processado.

#### 4.7.2.2 Escalabilidade: Speedup e Eficiência

A avaliação de escalabilidade com 1, 2, 4 e 8 *threads*, ilustrada na Figura 2.

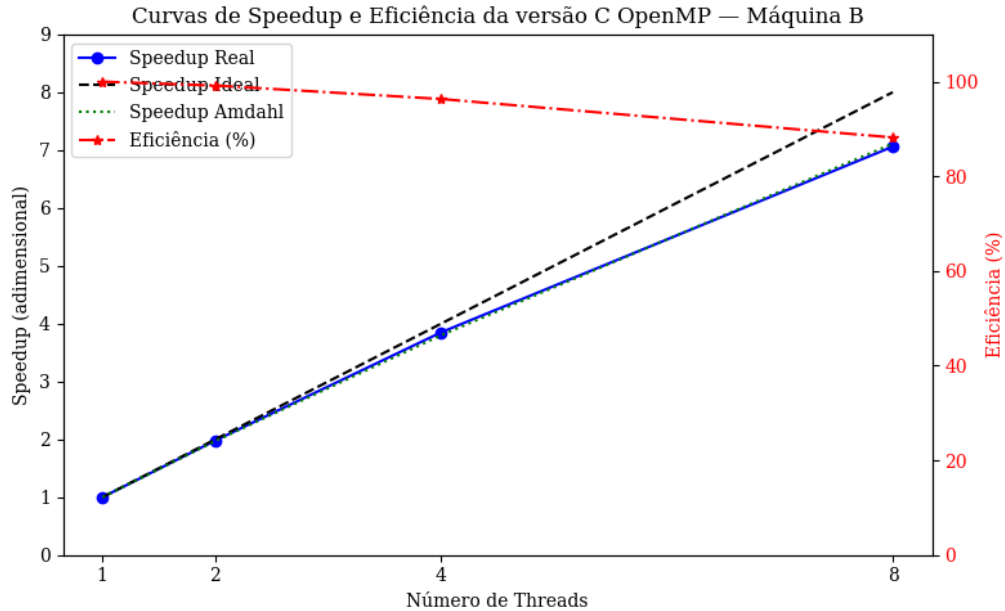


Figura 2 – Curvas de *Speedup* e Eficiência da versão C OpenMP – Máquina B.

Os valores calculados para a escalabilidade do código OpenMP na Máquina B foram sintetizados na Tabela 25.

Tabela 25 – Métricas de escalabilidade e comparação com a Lei de Amdahl na Máquina B.

| <i>Threads</i> ( $p$ ) | Tempo ( $T_p$ ) | <i>Speedup</i> Real ( $S_p$ ) | Amdahl ( $S_{\text{Amdahl}}$ ) | Eficiência ( $E_p$ ) |
|------------------------|-----------------|-------------------------------|--------------------------------|----------------------|
| 1                      | 31,13 s         | 1,00×                         | 1,00×                          | 100,00%              |
| 2                      | 15,70 s         | 1,98×                         | 1,97×                          | 99,19%               |
| 4                      | 8,08 s          | 3,85×                         | 3,80×                          | 96,34%               |
| 8                      | 4,41 s          | 7,06×                         | 7,11×                          | 88,22%               |

Fonte: Elaboração própria (2026).

A Máquina B demonstrou escalabilidade superior à Máquina A em todas as configurações, atingindo 88,22% de eficiência com 8 threads contra 75,86% da Máquina A. Esse comportamento decorre da topologia homogênea do i7-11800H: como todos os núcleos físicos são simétricos e operam sob a mesma frequência e vazão de instruções, o escalonador distribui as threads de forma uniforme sem as penalidades impostas pela assimetria entre P-cores e E-cores da Máquina A. O speedup real de 7,06× com 8 threads aproximou-se do limite teórico de Amdahl (7,11×), evidenciando aproveitamento quase ideal do paralelismo disponível.

## 4.8 Análise da Versão Python com Multithreading

A paralelização via módulo `threading` foi avaliada sob os mesmos parâmetros para analisar a capacidade de escalabilidade da linguagem sob concorrência em memória compartilhada.

### 4.8.1 Máquina A

#### 4.8.1.1 cProfile e perf

O *profiling* com *cProfile* registrou 571,03 s de execução, dos quais 524,65 s (91,87%) concentraram-se estritamente no cálculo do fractal (`check_mandelbrot`). As operações de sincronização e fila de tarefas somaram menos de 0,15 s, comprovando que o gargalo reside na execução do código interpretado. A Tabela 26 apresenta os dados de hardware coletados via `perf stat` para 4 *threads*.

Tabela 26 – Métricas de hardware da versão Python com 4 *threads*.

| Métrica         | P-cores (cpu_core)                                       | E-cores (cpu_atom)    |
|-----------------|----------------------------------------------------------|-----------------------|
| Ciclos          | 2,28 trilhões (99,27%)                                   | 1,57 trilhão (0,74%)  |
| Instruções      | 12,23 trilhões (99,27%)                                  | 6,04 trilhões (0,74%) |
| IPC             | 5,36                                                     | 3,83                  |
| Branch-misses   | 239.999.526                                              | 514.053.516           |
| LLC-load-misses | 6.597.355                                                | 19.271.483            |
| Tempo Total (s) | 522,55 s ( <i>user</i> : 522,44 s / <i>sys</i> : 7,55 s) |                       |

A amostragem do `perf report` indicou que o tempo total permaneceu rateado igualmente entre as quatro *threads* ( $\approx 25\%$  cada), competindo pelo laço de execução do interpretador enquanto acionavam chamadas de trava no nível do sistema operacional. Por imposição do *Global Interpreter Lock* (GIL), apenas uma *thread* executa instruções Python por vez, resultando em concorrência preemptiva em vez de paralelismo simultâneo real.

#### 4.8.1.2 Impacto do GIL: Speedup e Eficiência

A Tabela 27 e a Figura 3 apresentam a escalabilidade e o comportamento de trocas de contexto frente à execução serial ( $T_1 = 522,52$  s).

Tabela 27 – Métricas de escalabilidade e contenção da versão Python Multithreading.

| Threads ( $p$ ) | Tempo ( $T_p$ ) | Speedup ( $S_p$ ) | Eficiência ( $E_p$ ) | Trocas Vol. | Uso CPU |
|-----------------|-----------------|-------------------|----------------------|-------------|---------|
| 1               | 522,52 s        | 1,00×             | 100,00%              | 9           | 99%     |
| 2               | 515,92 s        | 1,01×             | 50,64%               | 316.683     | 100%    |
| 4               | 519,49 s        | 1,00×             | 25,14%               | 551.321     | 100%    |
| 8               | 526,44 s        | 0,99×             | 12,41%               | 967.626     | 102%    |

Diferente do OpenMP, o uso de *threads* no python *multithreading* resultou em ganho nulo de velocidade, com o *speedup* estagnado e uso de CPU fixo em 100%. As trocas de

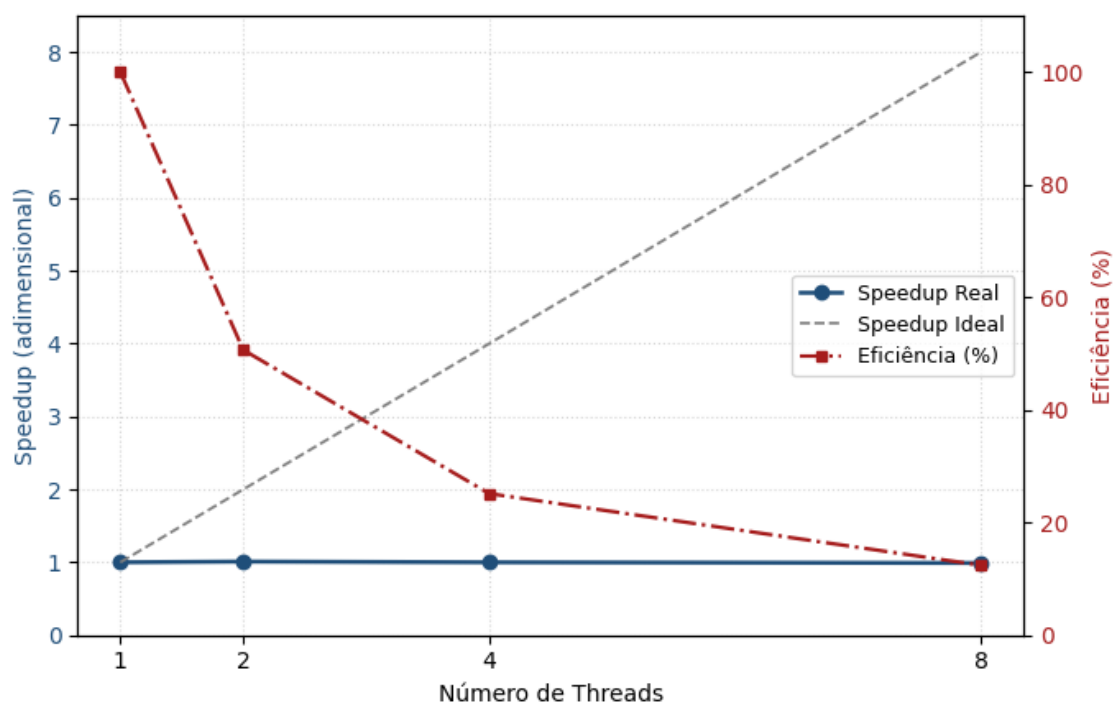


Figura 3 – Curvas de *speedup* e eficiência da versão Python com *multithreading*.

contexto voluntárias saltaram de 9 para quase 1 milhão com 8 *threads*, gerando sobrecarga adicional que reduziu a eficiência para 12,41% e provocou desaceleração sutil no tempo total.

## 4.8.2 Máquina B

### 4.8.2.1 cProfile

O cProfile evidenciou que 669,035 s dos 669,283 s totais foram consumidos pela primitiva `acquire` do lock do interpretador, invocada 10.266 vezes. As funções de gerenciamento de threads (`join`, `_wait_for_tstate_lock`) acumularam tempo cumulativo equivalente ao total da execução, confirmando que as threads passaram praticamente todo o tempo aguardando a liberação do GIL em vez de executar código útil.

#### 4.8.2.2 perf stat

Tabela 28 – Métricas de hardware da versão Python Multithreading com 4 threads na Máquina B.

| Métrica              | Python Serial       | Python Threading (4T) |
|----------------------|---------------------|-----------------------|
| Instruções           | 12.174.988.295.716  | 12.356.014.316.008    |
| Ciclos               | 3.287.253.055.502   | 3.150.721.295.376     |
| IPC                  | 3,70                | 3,92                  |
| Branch-misses        | 136.271.051 (0,01%) | 260.633.346 (0,01%)   |
| L1-dcache-misses     | 77.734.815          | 320.620.031           |
| LLC-load-misses      | 270.392             | 6.073.883             |
| Tempo Wall-clock (s) | 734,83              | 687,55                |

Fonte: Elaboração própria (2026).

As métricas foram muito semelhantes ao Python serial, confirmando que o GIL impediu execução paralela real. O aumento de L1-dcache-misses ( $4,12\times$ ) e LLC-load-misses ( $22,46\times$ ) decorre da contenção entre threads disputando as estruturas internas do interpretador na memória cache.

Tabela 29 – Principais funções identificadas via perf record no Python Multithreading (Máquina B).

| Self (%) | Função                                      | Origem     |
|----------|---------------------------------------------|------------|
| 77,66%   | <code>_PyEval_EvalFrameDefault</code>       | python3.11 |
| 12,40%   | <code>PyFloat_FromDouble</code>             | python3.11 |
| 1,31%    | <code>0x1aa842 (bytecode interno)</code>    | python3.11 |
| 0,36%    | <code>entry_SYSCALL_64_after_hwframe</code> | kernel     |
| 0,29%    | <code>__x64_sys_futex</code>                | kernel     |

Fonte: Elaboração própria (2026).

O `perf record` corroborou esse comportamento, com 86% das amostras concentradas em `_PyEval_EvalFrameDefault` — o laço principal de decodificação de bytecode do CPython — evidência direta da serialização imposta pelo GIL.

### 4.8.3 Máquina B

Tabela 30 – Métricas de escalabilidade e contenção da versão Python Multithreading na Máquina B.

| <i>Threads</i> ( $p$ ) | <i>Tempo</i> ( $T_p$ ) | <i>Speedup</i> ( $S_p$ ) | <i>Amdahl</i> | <i>Eficiência</i> ( $E_p$ ) |
|------------------------|------------------------|--------------------------|---------------|-----------------------------|
| 1                      | 730,23 s               | 1,00×                    | 1,00×         | 100,00%                     |
| 2                      | 693,19 s               | 1,05×                    | 1,97×         | 52,72%                      |
| 4                      | 672,91 s               | 1,08×                    | 3,80×         | 27,11%                      |
| 8                      | 677,24 s               | 1,08×                    | 7,11×         | 13,47%                      |

Fonte: Elaboração própria (2026).

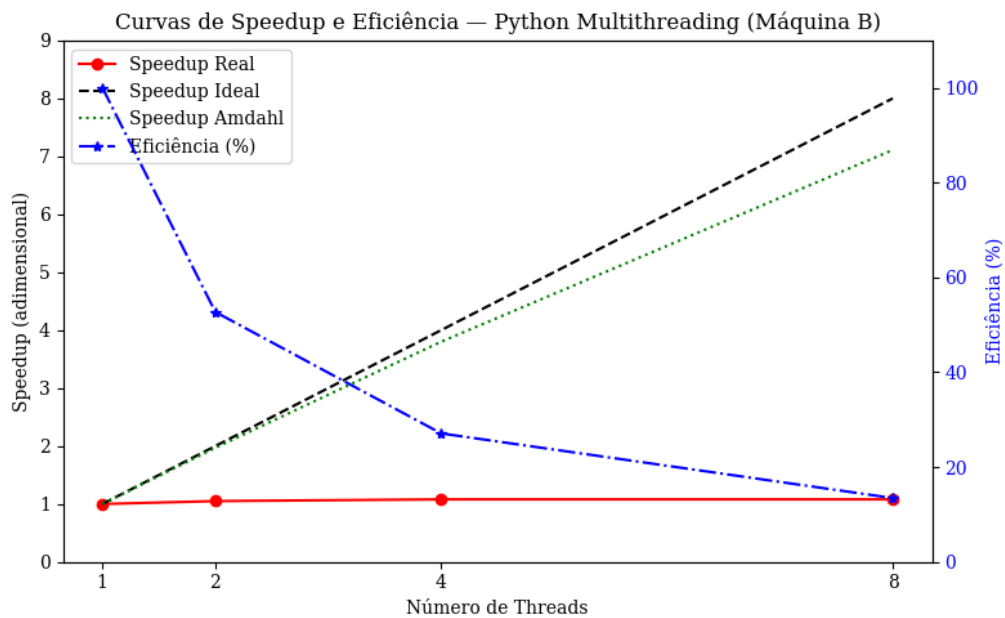


Figura 4 – Curvas de *Speedup* e Eficiência da versão Python Multithreading – Máquina B.

O speedup permaneceu estagnado em torno de  $1,08\times$  independentemente do número de threads, confirmando que o GIL serializa a execução do bytecode e impede paralelismo real em código CPU-bound. O comportamento é análogo ao da Máquina A, demonstrando que o impacto do GIL é independente da arquitetura de hardware.

Dessa maneira, fica confirmado que a versão multithreading não entrega paralelismo real, sendo mais indicado utilizar Multiprocess do Python ou, preferivelmente, OpenMP para trabalhar em contexto de memória compartilhada.

## 4.9 Análise da Versão Python com Multiprocessing

A paralelização via módulo `multiprocessing` isola os interpretadores em espaços de endereçamento próprios, contornando a trava global do Python e permitindo o uso simultâneo de múltiplos núcleos físicos.

#### 4.9.1 Máquina A

##### 4.9.1.1 cProfile e perf

O *profiling* do processo coordenador registrou 184,91 s de tempo total. A decomposição do tempo cumulativo indica que o despacho de dados e sincronização de linhas via canais de escrita respondeu por 147,03 s, ao passo que o bloqueio ativo aguardando o encerramento dos processos trabalhadores consumiu 37,44 s. Custos com instanciação via bifurcação de processos e manipulação de memória compartilhada mantiveram-se residuais, somando menos de 0,10 s. A Tabela 31 detalha a distribuição das métricas de hardware entre as classes de núcleos físicos.

Tabela 31 – Métricas de hardware da versão Python Multiprocessing com quatro processos.

| Métrica                           | Núcleos P               | Núcleos E             |
|-----------------------------------|-------------------------|-----------------------|
| Ciclos                            | 2,43 trilhões (95,15%)  | 1,88 trilhão (4,96%)  |
| Instruções                        | 12,61 trilhões (95,15%) | 7,18 trilhões (4,96%) |
| IPC                               | 5,19                    | 3,82                  |
| Branch-misses                     | 169.926.628             | 231.262.146           |
| LLC-load-misses                   | 15.379.732              | 82.659                |
| Tempo de Execução (s)             | 640,66 ( <i>user</i> )  | / 1,05 ( <i>sys</i> ) |
| Tempo Total ( <i>Wall-clock</i> ) |                         | 160,82 s              |

A amostragem de eventos do relatório do **perf** confirma a superação do gargalo de concorrência: mais de 90% das instruções executadas concentraram-se diretamente na avaliação de laços aritméticos nas instâncias paralelas, sem retenção expressiva em primitivas de sincronização do sistema operacional.

##### 4.9.1.2 Escalabilidade: Speedup e Eficiência

O ganho de vazão frente à execução monoprocesso de 531,33 s está sintetizado na Tabela 32 e na Figura 5.

O comportamento contrasta com o bloqueio observado na versão sob *multithreading*, demonstrando escalabilidade real com a taxa de ocupação de processador atingindo 797% na alocação máxima. Até quatro processos, o ganho de velocidade sustentou-se elevado em virtude do confinamento das tarefas nos núcleos de performance.

Tabela 32 – Escalabilidade e taxa de ocupação da versão Python Multiprocessing.

| Processos | Tempo    | Aceleração | Eficiência | Trocas Vol. | Ocupação CPU |
|-----------|----------|------------|------------|-------------|--------------|
| 1         | 531,33 s | 1,00×      | 100,00%    | 490         | 99%          |
| 2         | 267,15 s | 1,99×      | 99,44%     | 390         | 199%         |
| 4         | 151,60 s | 3,50×      | 87,62%     | 452         | 399%         |
| 8         | 107,37 s | 4,95×      | 61,86%     | 3.671       | 797%         |

A retração de rendimento verificada com oito processos decorre do transbordamento mandatório de instâncias para os núcleos de eficiência do processador, cujas frequências e taxas de instruções por ciclo são inferiores, aliadas ao aumento na sobrecarga de comunicação interprocessos para agregação dos blocos da matriz.

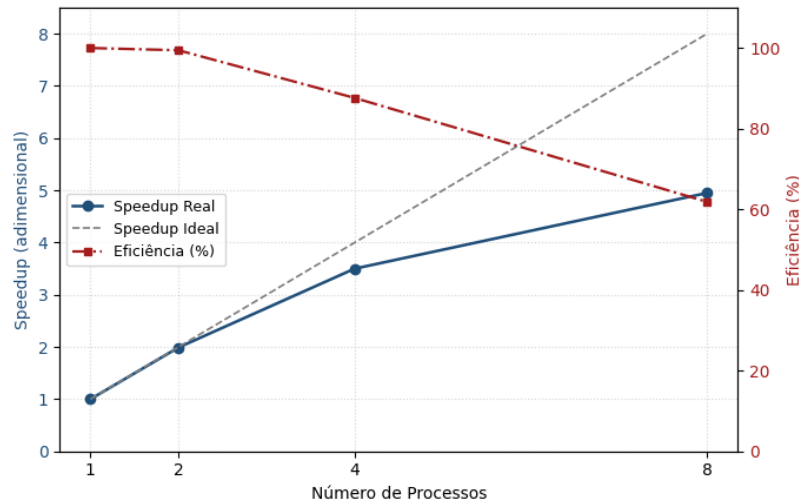


Figura 5 – Curvas de aceleração e eficiência da versão Python com *multiprocessing*.

## 4.9.2 Máquina B

### 4.9.2.1 cProfile e Perf

Tabela 33 – Métricas de hardware da versão Python Multiprocessing com 4 processos (perf stat)

| Métrica                           | Valor           |
|-----------------------------------|-----------------|
| Tempo Real ( <i>Wall-clock</i> )  | 199,28 s        |
| Tempo de CPU ( <i>User time</i> ) | 795,66 s        |
| Razão (CPU / Real)                | $\approx 4,00x$ |
| Instruções Executadas             | 12,65 trilhões  |
| Instruções por Ciclo (IPC)        | 4,04            |

Tabela 34 – Principais funções identificadas via perf record no Python Multiprocessing

| Overhead       | Função                                | Origem     |
|----------------|---------------------------------------|------------|
| $\sim 76,63\%$ | <code>_PyEval_EvalFrameDefault</code> | python3.11 |
| $\sim 12,36\%$ | <code>PyFloat_FromDouble</code>       | python3.11 |
| 0,00%          | Primitivas de Lock/Mutex              | Sistema    |

A análise das métricas de hardware (Tabela 33) comprova a eficácia do isolamento de memória: o tempo total de CPU de 795,66 s distribuído no tempo real de 199,28 s estabelece uma razão de 4:1, confirmando a execução paralela em quatro núcleos. O

diferencial decisivo deste modelo em relação ao *multithreading* é a ausência de concorrência pelas travas do GIL, o que viabiliza a execução real de código paralelo, dado que cada processo python possui seu próprio GIL, não disputando pelo acesso a thread do python.

#### 4.9.2.2 Escalabilidade: Speedup e Eficiência

#### 4.9.3 Escalabilidade: Speedup e Eficiência

O ganho de vazão frente à execução monoprocesso de 702,23s na Máquina B está sintetizado na Tabela 35 e na Figura 6.

Assim como observado na Máquina A, o isolamento de memória promovido pelo *multiprocessing* contornou a restrição do GIL, permitindo o uso simultâneo dos recursos computacionais. A taxa de ocupação do processador escalou de forma quase linear, atingindo 798% na alocação máxima com oito processos. Até quatro instâncias, a eficiência manteve-se em um patamar favorável de 84,39%, estabelecendo uma aceleração de 3,38×.

Tabela 35 – Escalabilidade e taxa de ocupação da versão Python Multiprocessing (Máquina B).

| Processos | Tempo    | Aceleração | Eficiência | Trocas Vol. | Ocupação CPU |
|-----------|----------|------------|------------|-------------|--------------|
| 1         | 702,23 s | 1,00×      | 100,00%    | 10.343      | 100%         |
| 2         | 366,18 s | 1,92×      | 95,89%     | 8.443       | 199%         |
| 4         | 208,02 s | 3,38×      | 84,39%     | 10.534      | 399%         |
| 8         | 130,83 s | 5,37×      | 67,09%     | 10.649      | 798%         |

A retração da eficiência para 67,09% com oito processos diverge do comportamento da Máquina A. Como a arquitetura da Máquina B possui oito núcleos simétricos de alta performance (sem divisão entre P-cores e E-cores), essa perda de rendimento não decorre de assimetria de hardware. O distanciamento do limite teórico de Amdahl nesse cenário atribui-se estritamente à saturação do barramento de memória durante o acesso concorrente ao bloco de *SharedMemory* e ao *overhead* do sistema operacional na coordenação e junção dos resultados finais via *Queue*.

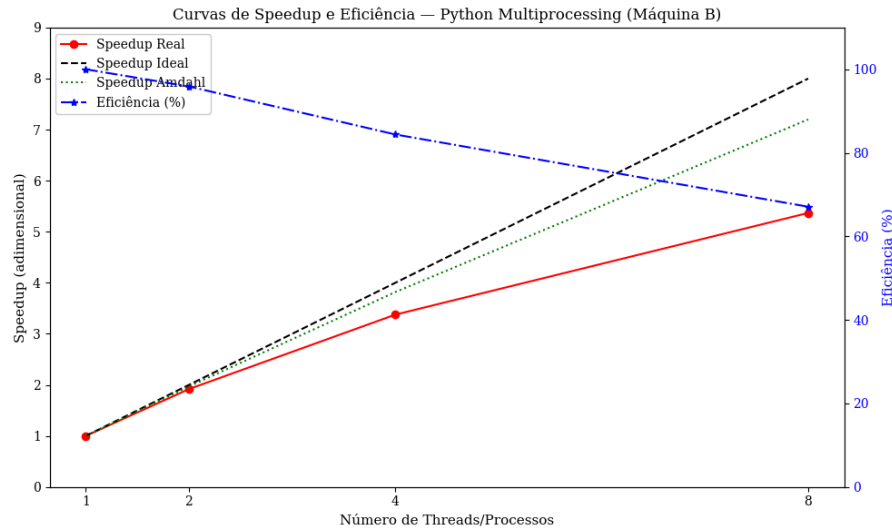


Figura 6 – Curvas de aceleração e eficiência da versão Python com *multiprocessing* (Máquina B).

## 4.10 Overhead de Processo e Memória

### 4.10.1 Máquina A

Embora o isolamento de instâncias contorne a trava global do interpretador, a criação de múltiplos processos acarreta custos estruturais de consumo de memória física e gerenciamento do sistema operacional.

Tabela 36 – Consumo de memória residente e faltas de página no Python Multiprocessing.

| Processos | Memória Máxima (RSS) | Minor Page Faults | Tempo em Kernel (sys) |
|-----------|----------------------|-------------------|-----------------------|
| 1         | 920,39 MB            | 244.331           | 0,62 s                |
| 2         | 920,44 MB            | 252.672           | 0,52 s                |
| 4         | 920,54 MB            | 253.892           | 0,62 s                |
| 8         | 920,54 MB            | 256.516           | 0,72 s                |

A Tabela 36 apresenta o impacto do particionamento de tarefas na utilização de recursos da Máquina A. Em contraste com a versão sob concorrência por *threads*, cujo consumo de memória manteve-se próximo de 461 MB, o arranjo multiprocesso demandou aproximadamente 920 MB de conjunto residente máximo (*RSS*). Esse acréscimo reflete a instanciação de interpretadores CPython completos para cada trabalhador e, primordialmente, a alocação do segmento de memória compartilhada utilizado para consolidar a imagem final de 15.360 por 10.240 pixels.

O comportamento de faltas de página secundárias (*minor page faults*) evidencia a atuação do mecanismo de cópia em escrita (*copy-on-write*) provido pela chamada de sistema `fork`. O número de faltas aumentou de 244.331 para 256.516 entre 1 e 8 processos, demonstrando que as páginas de memória do processo pai permaneceram compartilhadas até que os trabalhadores iniciassem a escrita dos resultados das linhas no mapa compar-

tilhado. O tempo despendido em *System time* manteve-se reduzido e estável em todas as configurações, confirmando que a sobrecarga do modelo concentra-se na ocupação de memória primária e não no custo de escalonamento do sistema operacional.

#### 4.10.2 Máquina B

Tabela 37 – Consumo de memória residente e faltas de página no Python Multiprocessing (Máquina B).

| Processos | Memória Máxima (RSS) | Minor Page Faults | Tempo em Kernel (sys) |
|-----------|----------------------|-------------------|-----------------------|
| 1         | 937,24 MB            | 240.539           | 0,56 s                |
| 2         | 939,03 MB            | 147.782           | 0,54 s                |
| 4         | 939,18 MB            | 149.740           | 0,50 s                |
| 8         | 939,20 MB            | 153.538           | 0,51 s                |

O comportamento na Máquina B Tabela 37 seguiu estritamente o padrão observado na Máquina A. O conjunto residente máximo (*RSS*) estabeleceu-se na faixa dos 939 MB, ratificando a pegada de memória superior intrínseca ao modelo de *multiprocessing* associado à alocação de blocos de *SharedMemory*.

### 4.11 Tabela Comparativa dos Três Modelos de Paralelização

#### 4.11.1 Máquina A

A consolidação dos ensaios empíricos expõe o contraste entre abordagens compiladas de baixo nível e abstrações interpretadas sob concorrência em memória compartilhada ou processos isolados. A Tabela 38 reúne os tempos de resposta, aceleração e consumo de memória primária para as três abordagens em 1, 2, 4 e 8 unidades de execução.

Tabela 38 – Comparativo de desempenho, aceleração e memória entre C e Python.

| Modelo                          | Fluxos ( $p$ ) | Tempo ( $T_p$ ) | Speedup ( $S_p$ ) | Memória (RSS) |
|---------------------------------|----------------|-----------------|-------------------|---------------|
| <b>C / OpenMP</b>               | 1              | 24,72 s         | 1,00×             | 461,2 MB      |
|                                 | 2              | 12,44 s         | 1,99×             | 461,2 MB      |
|                                 | 4              | 6,57 s          | 3,76×             | 461,3 MB      |
|                                 | 8              | 4,07 s          | 6,07×             | 461,4 MB      |
| <b>Python / Threading</b>       | 1              | 522,52 s        | 1,00×             | 461,5 MB      |
|                                 | 2              | 515,92 s        | 1,01×             | 461,6 MB      |
|                                 | 4              | 519,49 s        | 1,00×             | 461,5 MB      |
|                                 | 8              | 526,44 s        | 0,99×             | 461,7 MB      |
| <b>Python / Multiprocessing</b> | 1              | 531,33 s        | 1,00×             | 920,4 MB      |
|                                 | 2              | 267,15 s        | 1,99×             | 920,4 MB      |
|                                 | 4              | 151,60 s        | 3,50×             | 920,5 MB      |
|                                 | 8              | 107,37 s        | 4,95×             | 920,5 MB      |

Os resultados evidenciam a superioridade de desempenho da linguagem compilada nativa frente às abordagens interpretadas. A versão em C monothread ( $T_1 = 24,72$  s) foi

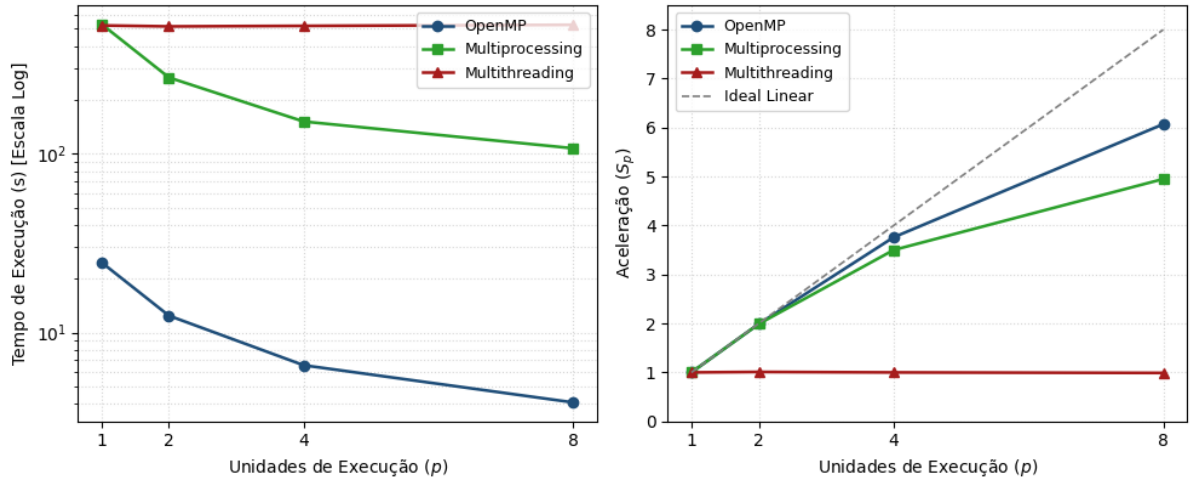


Figura 7 – Comparação de tempo de execução e aceleração entre C OpenMP e as abordagens Python.

4,34 vezes mais rápida do que a melhor configuração paralela em Python, obtida pelo módulo `multiprocessing` com 8 processos ( $T_8 = 107,37$  s). Quando confrontada com a execução serial em Python ( $T_1 = 531,33$  s), a implementação básica em C alcançou uma redução de tempo correspondente a um fator de 21,50 vezes.

No ecossistema Python, o paralelismo via `threading` mostrou-se ineficaz para cargas computacionais puramente delimitadas por CPU, mantendo o tempo estagnado em torno de 520 s em virtude da contenção imposta pelo *Global Interpreter Lock*. Já o módulo `multiprocessing` viabilizou paralelismo real em múltiplos núcleos ao instanciar interpretadores isolados, atingindo uma aceleração de 4,95 vezes com 8 processos. Essa escalabilidade, contudo, demandou o dobro de memória primária, elevando o conjunto residente de 461 MB para 920 MB devido à replicação do ambiente de execução e à alocação do segmento de memória compartilhada.

#### 4.11.2 Máquina B

Tabela 39 – Comparativo de desempenho, aceleração e memória entre C e Python (Máquina B).

| Modelo                          | Fluxos ( $p$ ) | Tempo ( $T_p$ ) | Speedup ( $S_p$ ) | Memória (RSS) |
|---------------------------------|----------------|-----------------|-------------------|---------------|
| <b>C / OpenMP</b>               | 1              | 31,13 s         | 1,00×             | 462,7 MB      |
|                                 | 2              | 15,70 s         | 1,98×             | 462,7 MB      |
|                                 | 4              | 8,08 s          | 3,85×             | 462,7 MB      |
|                                 | 8              | 4,41 s          | 7,06×             | 462,7 MB      |
| <b>Python / Threading</b>       | 1              | 730,23 s        | 1,00×             | 474,3 MB      |
|                                 | 2              | 693,19 s        | 1,05×             | 474,3 MB      |
|                                 | 4              | 672,91 s        | 1,08×             | 474,3 MB      |
|                                 | 8              | 677,24 s        | 1,08×             | 474,3 MB      |
| <b>Python / Multiprocessing</b> | 1              | 702,23 s        | 1,00×             | 939,2 MB      |
|                                 | 2              | 366,18 s        | 1,92×             | 939,2 MB      |
|                                 | 4              | 208,02 s        | 3,38×             | 939,2 MB      |
|                                 | 8              | 130,83 s        | 5,37×             | 939,2 MB      |

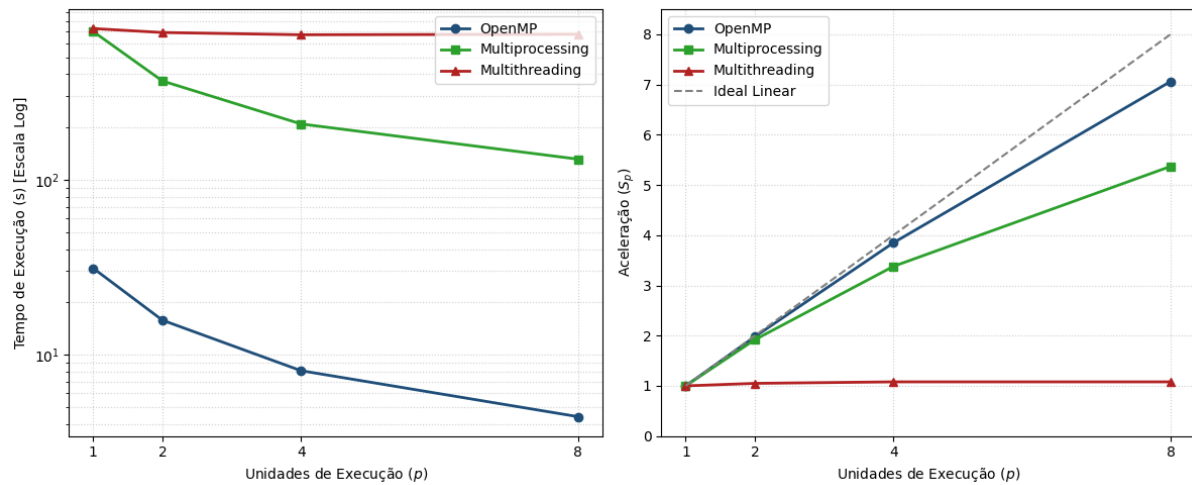


Figura 8 – Comparação de tempo de execução e aceleração entre C OpenMP e as abordagens Python (Máquina B).

A consolidação dos dados para a Máquina B (Tabela 39) replicou integralmente a dinâmica comportamental observada na Máquina A. A diferença limitou-se aos valores absolutos obtidos, reflexo direto da topologia do processador. O código nativo em C manteve sua expressiva vantagem, com uma redução de tempo de 22,55 vezes na comparação estritamente serial ( $T_1 = 31,13$  s contra  $T_1 = 702,23$  s do Python).

No cenário paralelo, o GIL confirmou-se como um gargalo arquitetural independente do *hardware*, inviabilizando novamente o ganho via **threading** (aceleração máxima de 1,08×). O **multiprocessing** foi a única abordagem interpretada a entregar paralelismo real (5,37×), exigindo, no entanto, o mesmo comportamento de replicação de memória primária (de 474,3 MB para 939,2 MB) para isolar os interpretadores.

## 4.12 Comparação C Serial versus Python Serial

O confronto direto entre as implementações sequenciais evidencia o impacto da sobrecarga estrutural do interpretador CPython frente à execução nativa em código de máquina compilado para a arquitetura x86-64.

### 4.12.1 Máquina A

#### 4.12.1.1 Tempo de Execução

Na avaliação com apenas um fluxo de processamento, a versão em C concluiu o cálculo do fractal em 24,72 s, enquanto a versão equivalente em Python requereu 522,52 s. Essa diferença estabelece uma razão de aceleração de 21,14 vezes em favor do código compilado, sob o mesmo conjunto de parâmetros matemáticos e limites de convergência.

Essa discrepância decorre das estratégias de execução de cada linguagem. No binário compilado em C, o laço de cálculo do pixel opera diretamente com variáveis escalares em registradores de ponto flutuante, sem camadas intermediárias de verificação. Em contrapartida, cada iteração em Python demanda o despacho e avaliação de *bytecodes*, a resolução dinâmica de tipos e o empacotamento contínuo de objetos numéricos com gerenciamento de contagem de referências.

#### 4.12.1.2 Métricas de Hardware

A Tabela 40 sintetiza os contadores de microarquitetura coletados via `perf stat` para os cenários com uma unidade de execução.

Tabela 40 – Métricas de microarquitetura: C Serial versus Python Serial.

| Métrica de Hardware   | C Serial      | Python Serial  | Razão (Python / C) |
|-----------------------|---------------|----------------|--------------------|
| Instruções Executadas | 77,54 bilhões | 12,23 trilhões | 157,72×            |
| Ciclos de CPU         | 78,85 bilhões | 2,28 trilhões  | 28,92×             |
| IPC Médio             | 0,98          | 5,36           | 5,47×              |
| Branch-misses         | 14,20 milhões | 239,99 milhões | 16,90×             |
| Faltas de Cache (LLC) | 1,82 milhão   | 6,59 milhões   | 3,62×              |
| Conjunto Residente    | 461,20 MB     | 461,50 MB      | 1,00×              |

Os dados de hardware detalham o comportamento do processador e esclarecem a inversão observada no IPC, no qual Python registra 5,36 instruções por ciclo contra 0,98 em C. O interpretador demanda 157,72 vezes mais instruções para processar o mesmo plano de pontos, mantendo os estágios de decodificação e execução saturados com rotinas internas da máquina virtual.

Embora o fluxo de instruções em Python apresente paralelismo em nível de instrução favorável à microarquitetura dos núcleos de performance, esse processamento é dominado

por código de suporte do próprio interpretador. A implementação em C necessita de uma fração das instruções totais para executar a mesma carga aritmética, reduzindo a demanda global de ciclos em 28,92 vezes e diminuindo substancialmente a pressão sobre os preditores de saltos e o subsistema de memória cache.

#### 4.12.1.3 Eficiência de Paralelização

A transição do paradigma sequencial para as versões paralelas expõe divergências fundamentais na forma como cada modelo aproveita os recursos da arquitetura heterogênea:

- **C com OpenMP:** Opera diretamente sobre os fios de execução do sistema operacional em espaço compartilhado de memória. A aceleração escala quase linearmente até quatro fluxos (3,76 vezes) e alcança 6,07 vezes em oito fluxos, com a retração de eficiência restrita às diferenças de capacidade entre os núcleos P e E.
- **Python com Multithreading:** Mostra-se inviável para ganhos de desempenho computacional em virtude da trava global do interpretador. O tempo de resposta permaneceu estagnado em torno de 520 s em todas as alocações, culminando em 12,41% de eficiência com oito fluxos e elevando as trocas voluntárias de contexto para a faixa de centenas de milhares.
- **Python com Multiprocessing:** Viabiliza paralelismo efetivo ao isolar interpretadores em processos independentes, atingindo aceleração de 4,95 vezes com oito processos. No entanto, essa abordagem dobra a ocupação de memória física para 920 MB e incorre em custos adicionais de comunicação interprocessos para montagem do arranjo final.

O ambiente compilado nativo converte o paralelismo diretamente em redução de tempo de execução sem penalidades de memória primária. Já o ambiente interpretado exige a replicação integral dos processos de execução para superar restrições de concorrência, transferindo para o sistema operacional a sobrecarga de gerenciar múltiplos espaços de memória e canais de sincronização.

#### 4.12.2 Máquina B

#### 4.12.3 Tempo de Execução

Tabela 41 – Comparativo de tempo de execução entre C Serial e Python Serial.

| Métrica de Tempo            | C Serial | Python Serial | Razão (Python/C) |
|-----------------------------|----------|---------------|------------------|
| <i>Wall-clock</i> (Total)   | 31,86 s  | 734,83 s      | 23,06×           |
| <i>User time</i> (CPU)      | 31,51 s  | 734,44 s      | 23,30×           |
| <i>System time</i> (Kernel) | 0,22 s   | 0,20 s        | 0,90×            |

A comparação direta dos tempos de execução (Tabela 41) evidencia a diferença de desempenho entre o código de máquina nativo e a interpretação de *bytecode*. A versão em Python demandou 734,83 segundos de tempo real para processar o mesmo plano complexo que a versão em C concluiu em 31,86 segundos, estabelecendo uma razão de lentidão de 23,06 vezes. Em ambas as implementações, o tempo de *kernel* (*system time*) manteve-se irrisório (próximo a 0,20 s), confirmando que a carga de trabalho do fractal de Mandelbrot é estritamente limitada pela CPU (*CPU-bound*) e não sofreu interferência de gargalos de entrada e saída.

#### 4.12.4 Métricas de Hardware (perf stat)

Tabela 42 – Métricas de microarquitetura: C nativo versus interpretador CPython.

| Métrica de Hardware        | C Serial       | Python Serial  | Razão (Py/C) |
|----------------------------|----------------|----------------|--------------|
| Instruções Executadas      | 221,15 bilhões | 12,17 trilhões | 55,05×       |
| Ciclos de CPU              | 145,51 bilhões | 3,28 trilhões  | 22,55×       |
| IPC (Instruções por Ciclo) | 1,52           | 3,70           | 2,43×        |
| Faltas de Cache L1         | 16,88 milhões  | 77,73 milhões  | 4,60×        |

O rastreamento de microarquitetura revela a origem física dessa discrepância de tempo. Para resolver matematicamente a mesma grade de pontos, a versão em Python executou 12,17 trilhões de instruções, um volume 55 vezes maior que as 221 bilhões de instruções processadas pelo binário em C. Esse inchaço massivo reflete o *overhead* estrutural da máquina virtual do CPython, que precisa decodificar o *bytecode*, resolver tipagem dinâmica e gerenciar a contagem de referências a cada iteração do laço.

Além disso, o *footprint* de memória do Python provocou um salto de 4,60 vezes nas faltas de cache L1. Enquanto a linguagem C opera o laço interno com tipos primitivos nativos alocados de forma contígua e otimizados nos registradores, o Python encapsula cada coordenada em estruturas `PyObject`. Esse encapsulamento adiciona metadados pesados, fragmentando a localidade espacial na memória e forçando a CPU a buscar dados em níveis mais lentos da hierarquia de cache.

#### 4.12.5 Eficiência de Paralelização

A escolha do paradigma define tanto o desempenho base quanto a capacidade de escalar em múltiplos núcleos.

O **C associado ao OpenMP** representa o cenário ideal para memória compartilhada: as *threads* atuam no mesmo espaço de endereçamento de forma nativa, garantindo aceleração quase linear sem consumo extra de RAM. No **Python**, a abordagem via **Multithreading** esbarra no *Global Interpreter Lock* (GIL).

Em problemas intensivos de CPU como o Mandelbrot, o GIL impede a execução simultânea do código, travando o *speedup* em  $\sim 1,0\times$  independentemente dos núcleos disponíveis. Para obter paralelismo real em Python, a única saída é o **Multiprocessing**, que contorna o GIL criando processos isolados. Contudo, essa solução traz severas penalidades ao sistema:

- **Consumo de Memória:** A ocupação de RAM multiplica-se a cada novo processo instanciado.
- **Sobrecarga de Comunicação:** O empacotamento de dados e o tráfego em filas adicionam um custo computacional que não existe no C.

Para a **Computação de Alto Desempenho (HPC)**, conclui-se que o Python nativo é ineficiente para cálculos densos. Seu uso ideal restringe-se a atuar como interface (orquestração) para bibliotecas rápidas compiladas em C ou C++, unindo a facilidade de desenvolvimento à alta performance do *hardware*.

## 5 Conclusão

Este trabalho avaliou o desempenho e a escalabilidade do cálculo do fractal de Mandelbrot sob os paradigmas compilado (C) e interpretado (Python), comparando as abordagens seriais e os modelos de paralelização via OpenMP, *multithreading* e *multiprocessing*.

A versão compilada em C superou a implementação equivalente em Python por um fator superior a 20 vezes em tempo de execução. A instrumentação de baixo nível via **perf stat** confirmou que essa diferença decorre da sobrecarga da máquina virtual CPython, que demandou mais de 50 vezes o número de instruções para decodificação de *bytecode* e elevou as perdas de cache L1, enquanto o código em C operou quase que exclusivamente sobre registradores nativos.

Dentre os modelos paralelos, o C com OpenMP assegurou a melhor escalabilidade com baixo consumo de memória, ao passo que o Python Multiprocessing viabilizou paralelismo real contornando o GIL, embora ao custo de duplicar a ocupação de memória RAM para acomodar múltiplos interpretadores. Em contrapartida, o Python Multithreading mostrou-se inviável para processamento intensivo de CPU, mantendo aceleração nula e gerando quase um milhão de trocas voluntárias de contexto decorrentes da disputa contínua pelo bloqueio da máquina virtual.

Por fim, o contraste entre as máquinas evidenciou que processadores com núcleos homogêneos alcançam aproveitamento próximo ao limite analítico da Lei de Amdahl, enquanto arquiteturas híbridas sofrem retração de eficiência quando a distribuição de carga transborda para núcleos de menor desempenho. Conclui-se que o código nativo compilado

aliado a modelos de memória compartilhada permanece indispensável para aplicações de alta performance.

## Referências

FOSTER, I. *Designing and Building Parallel Programs*. Addison-Wesley, 1995. Disponível em: <<https://www.mcs.anl.gov/~itf/dbpp/>>.

Intel Corporation. *11th Generation Intel Core Processors: Product Brief*. 2021. Disponível em: <<https://www.intel.com/content/www/us/en/products/docs/processors/core/11th-gen-core-h-series-brief.html>>. Acesso em: 15 set. 2026.

INTEL CORPORATION. *Alder Lake: An Architecture Overview of Intel's First Hybrid Architecture*. 2021. Disponível em: <<https://www.intel.com/content/www/us/en/developer/articles/technical/hybrid-architecture.html>>. Acesso em: 14 set. 2026.

Python Wiki. *Global Interpreter Lock*. 2024. Disponível em: <<https://wiki.python.org/moin/GlobalInterpreterLock>>. Acesso em: 15 set. 2026.